

Project Title

SmartBites – AI Powered Personalized Diet Planner

By

Saba Hafeez

Roll No: RBSIT-21-04

Haiqa Ghaffar

Roll No: RBSIT-21-03

Session: 2021-2025

Supervisor: Mr. Ghulam Ghos



Department of Information Technology, Bahauddin Zakariya

University Multan

May 2026

BAHAUDDIN ZAKARIYA UNIVERSITY, MULTAN

Department of Information Technology

SmartBites – AI Powered Personalized Diet Planner

(SmartBites Diet Planner)

Degree of Bachelor of Science in Information Technology

By

SABA HAFEEZ

Roll No: BSIT-21-04

HAIQA GHAFFAR

Roll No: BSIT-21-03

Session: 2021 – 2025

Supervisor: Mr. Ghulam Ghos



May 2026

Certificate of Approval

This is to certify that the project titled "**SmartBites-AI Powered Personalized Diet Planner**" submitted by **SABA HAFEEZ(RBSIT-21-04) & HAIQA GHAFAR(RBSIT-21-03)** in partial fulfillment of the requirements for the degree of Bachelor of Science in Information Technology has been examined and approved.

Name: Mr. Ghulam Ghos

Signature: _____

Date: _____

Name: Mr. Usman

Signature: _____

Date: _____

Declaration

I hereby declare that this project titled " SmartBites – AI Powered Personalized Diet Planner" is my own work and has not been submitted elsewhere for any other degree or qualification.

Signature: _____

Name: SABA HAFEEZ

Roll No: RBSIT-21-04

Date: _____

Signature: _____

Name: HAIQA GHAFAR

Roll No: RBSIT-21-03

Acknowledgements

All praise, thanks, glorification, and gratitude are for ALLAH Almighty. I achieved my goal by ALLAH's infinite mercy and grace. I say Alhamdulillah for HIS uncountable blessings. After ALLAH, thanks to last Prophet Muhammad (ﷺ) who guided us toward the right path. Beloved prophet (ﷺ) says "Education is equally important for men and women".

Very special and humble thanks to my father. He is my main supporter to continue the studies. My father instilled iron determination in me. I just want to say, he is great, he is genius. I also express special thanks to my mother. She always prayed for me and improves my confidence to continue the higher study. My mother has always been very caring about my health, studies, and career. I would like to pay my heartiest gratitude to my sister and brothers for being a great source of support and encouragement to complete this work. Especially, thanks to my brothers who guided and helped me all along the way.

I would like to express my sincere gratitude to my supervisor **Mr. Ghulam Ghos** for the continuous support of my research work, for their patience, motivation, enthusiasm, and immense knowledge. Their guidance helped me during all the time of research and writing of this thesis. I could not have imagined having a better advisor and mentor for my research work.

Secondly, I would like to say thanks to **Mam Aliya Bodla**, principal and director of Regional Institute of Allied Health Sciences and Higher Education, Mian Channu, whose support and concern towards students is highly valuable.

In addition, I would like to acknowledge important role of **Mr. Usman**, whose help and assistance remained a source of motivation for me throughout this task.

SABA HAFEEZ(RBSIT-21-04)

HAIQA GHAFAR(RBSIT-21-03)

Abstract

This paper presents SmartBites – AI-Powered Personalized Diet Planner. Maintaining a balanced diet and healthy eating habits is increasingly challenging in today's fast-paced lifestyle due to time constraints and lack of personalized guidance. The SmartBites – AI-Powered Personalized Diet Planner project addresses this issue by providing intelligent and customized diet plans tailored to individual users. The system collects essential user data such as age, weight, height, activity level, and dietary preferences, and uses AI algorithms and nutritional analysis to generate personalized meal recommendations. The project comprises multiple functional modules including user authentication, profile management, diet plan generation, nutritional analysis, and chatbot interaction, ensuring secure login, profile updates, diet management, nutrition tracking, and real-time guidance through an AI-driven chatbot. The backend is implemented using Python FastAPI with MySQL for efficient data storage and retrieval, while the frontend employs HTML, CSS, JavaScript, and Bootstrap to provide a responsive and interactive interface. SmartBites delivers diet plans aligned with users' health goals and provides nutritional insights to support informed decisions. By integrating AI with nutritional science, the system ensures recommendations are personalized and adaptive, while the chatbot enhances user engagement. Overall, the project demonstrates a practical and intelligent approach to personalized nutrition management, offering accurate, data-driven, and actionable dietary guidance to support healthier lifestyles and long-term wellness goals.

Project Brief

PROJECT NAME	SmartBites – AI-Powered Personalized Diet Planner (Web App)
SUPERVISED BY	Mr. Ghulam Ghos
UNDERTAKEN BY	Saba Hafeez, Haiqa Ghaffar
DATE OF STARTING	15/02/ 2026
DATE OF COMPLETION	01/ 05 / 2026
OPERATING SYSTEM	Microsoft Windows 10 / 11
SOURCE LANGUAGE	HTML5, CSS3, JavaScript, Python 3.11+, FastAPI
DBMS USED	MySQL 9.4
TOOLS/PACKAGES	Visual Studio Code, XAMPP, Postman, Git, Uvicorn
SYSTEM	Intel(R) Core i5 / i7 CPU @ 2.50 GHz or above, 8.00 GB RAM, x64-based processor

List Of Abbreviations

Abbreviation	Full Form	Used In / Meaning
API	Application Programming Interface	Communication between frontend, backend, and AI services
JWT	JSON Web Token	User authentication and authorization
SQL	Structured Query Language	Database queries and operations
ORM	Object Relational Mapping	Converts Python objects into database tables
CRUD	Create, Read, Update, Delete	Basic database operations
BMI	Body Mass Index	User health measurement
BMR	Basal Metabolic Rate	Calories burned at rest
TDEE	Total Daily Energy Expenditure	Daily calorie requirement calculation
FastAPI	Fast Application Programming Interface	Python backend framework
SQLAlchemy	SQL Toolkit and ORM	Database management in Python
MySQL	My Structured Query Language	Relational database system
bcrypt	Blowfish Cryptographic Hashing	Password hashing
Groq	Groq AI Platform	AI inference provider
LLM	Large Language Model	AI chatbot model
FPDF	Free PDF	Python library for PDF generation
venv	Virtual Environment	Isolated Python environment
pip	Preferred Installer Program	Python package installer

TABLE OF CONTENTS

CHAPTER 1: INTRODUCTION	1
1.1 Introduction	1
1.2 Background of the Project	2
1.3 Problem Statement	3
1.4 Solution Overview	4
1.5 Objectives of the Project	5
1.6 Scope of the Project	7
1.7 Methodology	9
1.8 Document Structure	10
Chapter 2: Literature Review	12
2.1 Introduction	12
2.2 Evolution of Diet Planning Systems	12
2.3 Comparative Analysis of Existing Systems	13
2.4 Key Algorithms and Formulas in Diet Planning	15
Chapter 3: Requirements and Specifications	17
3.1 Introduction	17
3.2 Functional Requirements	17
3.3 Non-Functional Requirements	18
3.4 Hardware and Software Requirements	19
3.4.1 Hardware Requirements	19
3.5 Use Case Specifications	21
3.5.1 Use Case: User Registration	21
Chapter 4: System Architecture	26
4.1 Introduction	26
4.2 Architectural Design Approach	26
4.3 System Context Diagram	27

4.4 High-Level System Architecture	28
4.5 Detailed Module Descriptions	29
4.6 Database Design	30
4.6.1 Entity Relationships	31
4.7 Data Flow Description	33
4.8 Sequence diagram	33
4.8 Security Architecture	34
Chapter 5: Implementation	41
5.1 Introduction	41
5.2 Development Environment Setup	41
5.3 Technology Stack	41
5.4 Frontend Implementation	42
5.5 Backend Implementation	44
5.6 Database Implementation	47
5.7 AI Integration — Groq API with LLaMA 3.3 70B	49
5.8 User Onboarding — 6-Step Form	51
5.9 Diet Plan Generation Algorithm	52
5.10 Smart Features Implementation	55
5.11 Application Page Flow	56
5.12 Dashboard Implementation	58
5.13 Generated Diet Plan	58
5.14 Progress Tracking	59
5.15 Chat with Dr	59
5.16 Meal Logging and Calorie Gauge	59
5.17 PDF Report Generation	60
Chapter 6: Testing	61
6.1 Introduction	61
6.2 Testing Strategy	61
6.3 Unit Testing	62

6.4 Functional Testing-----	65
6.5 Integration Testing-----	69
6.6 Security Testing-----	73
6.7 Usability Testing-----	74
6.8 Cross-Browser Compatibility Testing -----	75
Chapter 7: Results and Benchmarks -----	77
7.1 Introduction-----	77
7.2 System Performance Results -----	77
7.3 AI Chatbot Response Quality-----	78
7.4 Key Achievements -----	80
Chapter 8: Conclusion and Future Work-----	81
8.1 Introduction-----	81
8.2 Conclusion-----	81
8.3 Limitations-----	82
8.4 Future Work-----	82
8.5 Final Remarks-----	83

List of Tables

Table 1.1: Key Problem Areas in Diet Planning-----	3
Table 1.2: Smart Bites Solution Features -----	5
Table 1.3: Project Objectives Summary-----	7
Table 1.4: Project Scope Boundary -----	8
Table 2.1: Evolution of Diet Planning Systems -----	13
Table 2.2: Comparative Analysis of Diet Planning Systems-----	15
Table 2.3: Activity Level Multipliers for TDEE Calculation -----	16
Table 2.4: Macronutrient Distribution by User Goal-----	16
Table 4.1: SmartBites Database Schema Overview -----	31
Table 5.1: SmartBites RESTful API Endpoints -----	46
Table 5.2: Activity Level Multipliers for TDEE Calculation -----	53
Table 5.3: Goal-Based Macronutrient Distribution-----	54
Table 5.4: Weight-Loss Mode Calorie Distribution -----	55
Table 6.1: Testing Strategy Overview-----	62
Table 6.2: Unit Test Cases and Results -----	64
Table 6.3: Functional Test Cases and Results -----	67
Table 6.4: Integration Test Cases and Results -----	70
Table 6.5: Security Test Cases and Results -----	74
Table 6.6: Usability Testing Results -----	75
Table 7.1: System Performance Benchmarks-----	78
Table 7.2: AI Chatbot Response Quality Evaluation -----	79

List of Figures

Figure 1.1: SmartBites System Development — Waterfall Model -----	9
Figure 1.2: Document Structure Flow Chart -----	11
Figure 3.1: Use Case — User Registration -----	21
Figure 3.2: Use Case — Generate Meal Plan -----	22
Figure 3.3: Use Case — PDF Report Generation -----	23
Figure 3.4: Use Case — AI Chatbot Interaction -----	24
Figure 3.5: Use Case — Nutritional Analysis -----	25
Figure 4.1: System Context Diagram -----	27
Figure 4.2: High-Level System Architecture -----	28
Figure 4.3: ER Diagram -----	32
Figure 4.4: Data Flow Diagram (DFD) -----	33
Figure 4.5: Sequence Diagram — Core Flow -----	34
Figure 4.6: Activity Diagram — SmartBites User Activities -----	36
Figure 4.7: Landing Page — Hero Section -----	38
Figure 5.1: Technology Stack Architecture -----	42
Figure 5.2: BMI Calculator on Landing Page -----	44
Figure 5.3: FastAPI Route Registration (main.py) -----	46
Figure 5.4: Login Page — Authentication -----	47
Figure 5.5: AI Coach — Groq API Integration (routes/ai.py) -----	50
Figure 5.6: AI Coach Chat Interface -----	50
Figure 5.7: Diet Engine Algorithm (calculator.py) -----	52
Figure 5.8: Dashboard — Diet Plan View -----	58
Figure 6.1: Pytest Execution Results -----	64
Figure 6.2: User Registration Page (FT-01) -----	68
Figure 6.3: User Login Page (FT-03) -----	68
Figure 6.4: Swagger — Register Request (IT-01) -----	71
Figure 6.5: Registration API Response (IT-01) -----	71
Figure 6.6: Swagger — Login Request (IT-02) -----	72
Figure 6.7: Login API Response (IT-02) -----	72

CHAPTER 1: INTRODUCTION

1.1 Introduction

In today's fast-paced world, maintaining a healthy and balanced diet has become increasingly challenging. People are often overwhelmed by conflicting nutrition advice, busy schedules, and the complexity of understanding their unique dietary needs. The traditional approach to diet planning — relying on generic charts, word-of-mouth advice, or occasional consultations with nutritionists — fails to account for individual differences in body composition, activity levels, medical conditions, and personal preferences.

The rapid advancement of artificial intelligence (AI) and web technologies has opened new possibilities for creating intelligent, personalized health management systems. AI-powered diet planners can analyze user data in real time, generate customized meal plans, track nutritional intake, and adapt recommendations based on progress — all without requiring face-to-face consultations with healthcare professionals.

Smart Bites is an AI-powered diet planning web application designed to address these challenges. It provides users with a comprehensive platform where they can input their personal information — including age, height, weight, activity level, dietary restrictions, and fitness goals — and receive a fully customized daily meal plan. The system leverages intelligent algorithms to calculate calorie requirements, distribute macronutrients, suggest meal timings, and offer alternative food options based on user preferences and availability.

Beyond simple meal generation, Smart Bites incorporates progress tracking features that allow users to log their daily food intake, monitor weight changes over time, and receive updated diet recommendations as their body metrics evolve. The platform also supports downloadable PDF diet charts, enabling users to carry their plans offline for convenient reference.

This project report documents the complete lifecycle of the Smart Bites system — from initial concept and requirements analysis through system design, implementation, testing, and evaluation. The report is organized into eight chapters following a structured four-phase approach: Foundation, Design, Development & Results, and Finalization.

1.2 Background of the Project

Diet planning has evolved significantly over the past few decades. In the past, individuals relied primarily on manual methods — consulting printed diet charts, following advice from friends and family, or seeking guidance from health magazines and online articles. These approaches, while accessible, suffered from several limitations: they were generalized, did not account for individual metabolic differences, and often led to inconsistent or unsustainable eating habits.

The emergence of mobile health (mHealth) applications in the early 2010s marked a turning point. Apps like MyFitnessPal, Lose It!, and Noom introduced digital food logging, calorie counting, and basic nutritional tracking to millions of users worldwide. However, most of these tools required significant manual input from users, offered limited personalization, and rarely adapted recommendations based on changing health metrics or user feedback.

The integration of artificial intelligence into healthcare and wellness platforms has further transformed the landscape. AI-driven systems can now process vast amounts of nutritional data, learn from user behavior patterns, and generate highly personalized recommendations that evolve over time. Machine learning algorithms can predict dietary needs, identify nutritional deficiencies, and suggest optimal meal combinations based on individual health profiles.

In Pakistan, the adoption of AI-based health tools remains in its early stages. While global platforms exist, they often lack support for local food databases, regional dietary preferences, and culturally relevant meal suggestions. Many users in the country still rely on traditional methods or generic online calculators that provide one-size-fits-all recommendations. The cost and availability of professional nutritionists further limit access to personalized dietary guidance, particularly for students and young professionals.

Smart Bites was conceived to bridge this gap — providing an affordable, accessible, and intelligent diet planning solution that caters to the specific needs of its users. By combining modern web technologies with AI-powered algorithms, the system delivers a level of personalization and convenience that was previously available only through expensive professional consultations.

1.3 Problem Statement

Table 1.1: Key Problem Areas in Diet Planning

Sr. No.	Problem Area	Description	Impact
1	Generic Diet Plans	One-size-fits-all charts ignore individual needs	Poor health outcomes
2	Manual Calorie Counting	Tedious and error-prone manual calculations	Inaccurate dietary tracking
3	Lack of Progress Tracking	No automated monitoring of weight or intake	Inability to adjust plans
4	Data Fragmentation	Records spread across paper, apps, spreadsheets	Data loss and inconsistency
5	High Cost of Experts	Professional nutritionists are expensive and not always accessible	Limited access to personalized guidance
6	No Real-Time Adaptation	Static plans cannot adjust to lifestyle changes	Unsustainable diet adherence

1.4 Solution Overview

Smart Bites addresses the identified problems by providing a comprehensive, AI-powered web-based diet planning platform. The system offers an end-to-end solution that covers every aspect of personalized diet management — from initial profile setup and goal definition to meal generation, daily tracking, progress monitoring, and plan adaptation.

The core features of the Smart Bites solution include:

Feature	Description	Benefit
Personalized Plans	AI generates customized diet plans based on user profile, goals, body type, age, and activity level	Better health outcomes tailored to individual needs
Automated Calorie Management	System automatically calculates daily calorie requirements and macro distribution	Eliminates manual calculation errors
Flexible Adjustments	Diet plans dynamically adapt based on user progress, feedback, and routine changes	Long-term sustainability and adherence
Progress Tracking	Visual charts and logs for daily intake, weight changes, and goal milestones	Keeps users informed and motivated
Alternative Suggestions	AI recommends substitute foods based on preferences, allergies, or availability	Greater flexibility in meal choices

Downloadable Reports	PDF diet charts and progress reports for offline use	Convenient access and sharing with professionals
Secure Data Storage	Encrypted storage of personal and health information	User privacy and data protection

Table 1.2: Smart Bites Solution Features

1.5 Objectives of the Project

1.5.1 General Objective

The primary objective of the Smart Bites project is to design, develop, and deploy an intelligent, AI-powered web-based application that automates and simplifies the process of personalized diet planning, making it accessible to users anytime and from any device connected to the internet.

1.5.2 Specific Objectives

- To design and develop an intelligent AI-based system that creates personalized diet plans using user profiles, activity levels, dietary preferences, and specific health goals.
- To provide instant access to customized meal plans, eliminating the need for manual calorie calculations or frequent expert consultations.
- To support flexible meal options and real-time diet adjustments based on user progress, feedback, and changing lifestyle patterns.
- To develop a simple and user-friendly interface that enables easy food logging, weight tracking, and continuous progress monitoring.
- To generate downloadable diet charts and reports in PDF format for offline reference, sharing, and long-term record keeping.
- To minimize time and effort in diet planning while improving accuracy, efficiency, and personalization through automation.

- To build a comprehensive diet management platform that integrates diet generation, progress tracking, and historical data management within a single system.
- To ensure secure storage and handling of user personal and health data through proper encryption and authentication mechanisms.
- To implement an admin panel for managing food databases, dietary rules, and system configuration parameters.

Objective ID	Description	Type	Priority
OBJ-01	AI-based personalized diet plan generation	Functional	High
OBJ-02	Instant customized meal plan access	Functional	High
OBJ-03	Real-time diet adjustments based on progress	Functional	High
OBJ-04	User-friendly interface for food logging and tracking	Functional	High
OBJ-05	Downloadable PDF diet charts and reports	Functional	Medium
OBJ-06	Automated calorie and macro calculations	Functional	High

OBJ-07	Comprehensive single-platform diet management	Functional	High
OBJ-08	Secure user data storage and authentication	Non-Functional	High

Table 1.3: Project Objectives Summary

1.6 Scope of the Project

This project focuses on designing and implementing an AI-powered personalized diet planner as a web-based application. The scope encompasses the following areas:

1.6.1 In-Scope

- User registration, authentication, and profile management with personal health data input.
- AI-powered diet plan generation based on user profiles including age, gender, weight, height, activity level, dietary preferences, and fitness goals (weight loss, weight gain, or maintenance).
- Daily meal plan display with meal timings, calorie counts, macronutrient breakdowns (protein, carbohydrates, fats), and portion sizes.
- Alternative food suggestion engine that recommends substitute meals based on user preferences, allergies, or food availability.
- Food intake logging and daily diet tracking functionality.
- Weight progress monitoring with visual charts and historical data views.
- Downloadable PDF diet charts and weekly/monthly progress reports.
- Admin dashboard for managing food databases, dietary rules, nutritional data, and user accounts.
- Responsive web design for access across desktops, tablets, and smartphones.

- Secure data storage with encrypted user information and HTTPS-based communication.

1.6.2 Out of Scope

- Integration with wearable fitness devices (e.g., smartwatches, fitness bands).
- Native mobile application development (iOS/Android) — the system is web-based only.
- Medical diagnosis or treatment recommendations — the system is for general diet planning, not clinical nutrition therapy.
- Real-time video consultation with nutritionists or healthcare providers.
- E-commerce or food delivery integration.
- Multi-language support beyond English in the current version.

Category	In Scope	Out of Scope
Platform	Web-based (responsive)	Native mobile apps
Diet Planning	AI-powered personalized meal plans	Medical/clinical nutrition therapy
Tracking	Food logging, weight monitoring, progress charts	Wearable device integration
Reports	PDF diet charts, progress summaries	Printed physical reports
Users	General public seeking diet management	Patients requiring clinical supervision
Languages	English	Multi-language support

Table 1.4: Project Scope Boundary

1.7 Methodology

For the development of Smart Bites, the Waterfall Model has been adopted as the primary software development methodology. The Waterfall Model is a structured, sequential approach where each phase of development — requirement analysis, system design, implementation, testing, and deployment — must be completed before the next phase begins. This linear progression ensures thorough documentation, systematic development, and clear milestone tracking.

The Waterfall methodology was selected for the following reasons:

- **Well-Defined Requirements:** The project requirements for Smart Bites are clearly defined from the outset, making a sequential approach practical and efficient.
- **Structured Documentation:** Each phase produces comprehensive documentation that serves as a reference for subsequent phases and future maintenance.
- **Predictable Timeline:** The phase-based structure allows for accurate project scheduling and resource allocation.
- **Quality Assurance:** Formal review and approval at each phase boundary ensures quality standards are maintained throughout development.
- **Ease of Management:** The linear nature of the model simplifies project management, progress tracking, and stakeholder communication

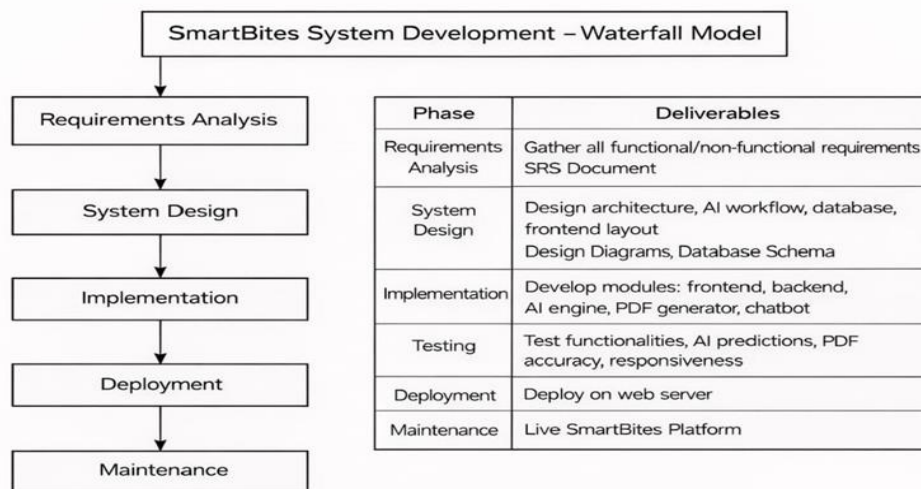


Figure 1.1: SmartBites System Development — Waterfall Model

1.8 Document Structure

This document is organized into eight chapters, each addressing a specific phase of the SmartBites project lifecycle — from initial research and planning to final evaluation and conclusion. The following structure provides a clear roadmap of the report:

Chapter 1 This chapter provides an overview of the SmartBites system, including its background, purpose, and the problem it aims to solve. It also introduces the motivation behind the project and gives a brief idea of the proposed solution.

Chapter 2 This chapter reviews existing systems and technologies related to diet planning and nutrition. It analyzes their limitations and highlights the research gaps that led to the development of SmartBites.

Chapter 3 This chapter defines the system requirements and specifications. It explains user needs, functional features, and technical requirements necessary for building the SmartBites system.

Chapter 4 This chapter describes the overall system architecture and design. It explains how different components such as frontend, backend, database, and AI modules interact with each other.

Chapter 5 This chapter focuses on the implementation of the system. It explains how the application is developed using selected technologies and how different modules are integrated.

Chapter 6 This chapter presents the testing process of the system. It includes test cases, validation methods, and ensures that the system meets all specified requirements.

Chapter 7 This chapter discusses the results obtained after implementation. It evaluates system performance and analyzes how effectively the system achieves its objectives.

Chapter 8 This chapter concludes the project by summarizing key outcomes and findings. It also suggests future improvements and enhancements for further development.

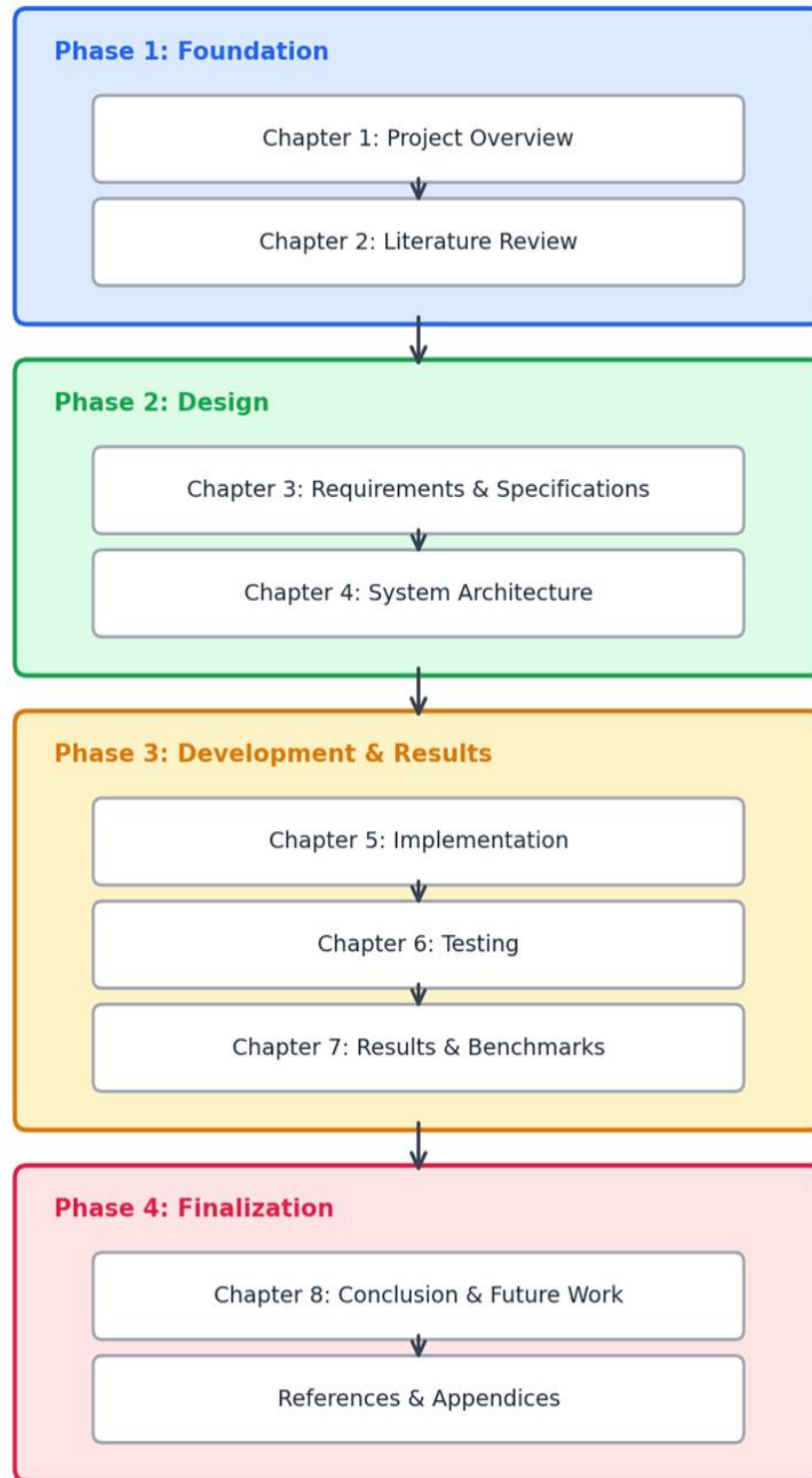


Figure 1.2: Document Structure Flow Chart

Chapter 2: Literature Review

2.1 Introduction

This chapter presents a comprehensive review of existing literature, research, and technologies related to AI-powered diet planning and nutrition management systems. The review examines the evolution of diet planning approaches, current state-of-the-art solutions, the role of artificial intelligence in personalized nutrition, and the gaps that Smart Bites aims to address. Understanding the strengths and limitations of existing systems is essential for designing a solution that offers genuine improvements in accuracy, usability, and personalization.

2.2 Evolution of Diet Planning Systems

The field of diet planning has undergone significant transformation over the past several decades. Early approaches were predominantly manual, relying on standardized food pyramids, recommended daily allowances (RDAs), and printed dietary guidelines published by government health agencies. These guidelines provided generalized recommendations for entire populations but failed to account for individual variations in metabolism, body composition, health conditions, and personal preferences.

The first generation of digital diet tools emerged in the late 2000s with web-based calorie counters and food databases. Websites like CalorieKing and FatSecret allowed users to search for food items and view their nutritional content. However, these tools required users to manually look up every food item, calculate totals, and compare against their targets — a process that was tedious and unsustainable for most users.

The mobile health revolution beginning around 2012 brought more sophisticated tools to the market. Applications like MyFitnessPal, Lose It!, and Cronometer introduced barcode scanning, extensive food databases, social features, and integration with fitness trackers. These apps significantly reduced the friction of food logging but still relied heavily on user input for accuracy. Studies have shown that manual food logging suffers from underreporting, portion size estimation errors, and declining engagement over time (Carels et al., 2005; Burke et al., 2011).

The current generation of diet planning systems leverages artificial intelligence, machine learning, and natural language processing to automate and personalize the diet planning process. AI-driven platforms can analyze user data patterns, predict nutritional needs, generate meal suggestions, and adapt recommendations in real time based on user feedback and progress metrics.

Era	Approach	Key Characteristics	Limitations
Pre-2000s	Manual / Paper-Based	Printed charts, RDAs, food pyramids	Generalized, no personalization
2000-2010	Web-Based Databases	Online food lookup, calorie tables	Manual data entry, tedious
2010-2018	Mobile Health Apps	Barcode scanning, food logging, fitness sync	User-dependent accuracy, engagement decline
2018-Present	AI-Powered Systems	ML recommendations, auto meal plans, progress adaptation	Data privacy concerns, algorithmic bias
Future	Genomic/IoT Integration	DNA-based nutrition, smart kitchen, real-time biosensors	High cost, regulatory challenges

Table 2.1: Evolution of Diet Planning Systems

2.3 Comparative Analysis of Existing Systems

The following table presents a detailed comparison of the reviewed diet planning systems against the features offered by Smart Bites:

Feature	MyFitnessPal	Noom	Eat This Much	Lifesum	Smart Bites
AI-Generated Meal Plans	No	No	Partial	No	Yes
Personalized Diet Plans	No	Limited	Yes	Pre-designed	Yes (AI-based)
Calorie Tracking	Yes	Yes	Yes	Yes	Yes
Macro Breakdown	Yes	Limited	Yes	Yes (Premium)	Yes
Progress Tracking	Yes	Yes	Limited	Yes	Yes
Alternative Food Suggestions	No	No	Limited	No	Yes (AI-based)
Dynamic Plan Adjustment	No	No	No	No	Yes
PDF Reports	No	No	No	No	Yes
Free Access	Freemium	Subscription	Freemium	Freemium	Free
Local Food Database	Partial	Limited	No	Limited	Yes

Web-Based Access	Yes	Yes	Yes	Yes	Yes
-------------------------	-----	-----	-----	-----	-----

Table 2.2: Comparative Analysis of Diet Planning Systems

2.4 Key Algorithms and Formulas in Diet Planning

The foundation of any AI-based diet planning system lies in accurate calorie and nutrient calculations. The following are the key formulas and algorithms that form the computational backbone of Smart Bites:

Basal Metabolic Rate (BMR) Calculation — Mifflin-St Jeor Equation:

For Males: $BMR = (10 \times \text{weight in kg}) + (6.25 \times \text{height in cm}) - (5 \times \text{age in years}) + 5$

For Females: $BMR = (10 \times \text{weight in kg}) + (6.25 \times \text{height in cm}) - (5 \times \text{age in years}) - 161$

Total Daily Energy Expenditure (TDEE) is calculated by multiplying BMR with an activity factor:

Activity Level	Description	Multiplier
Sedentary	<i>Little or no exercise, desk job</i>	<i>1.2</i>
Lightly Active	<i>Light exercise 1-3 days/week</i>	<i>1.375</i>
Moderately Active	<i>Moderate exercise 3-5 days/week</i>	<i>1.55</i>
Very Active	<i>Hard exercise 6-7 days/week</i>	<i>1.725</i>
Extra Active	<i>Very hard exercise, physical job</i>	<i>1.9</i>

Table 2.3: Activity Level Multipliers for TDEE Calculation

Macronutrient Distribution: Based on the calculated TDEE and user goals, the system distributes calories across three macronutrients using the following standard ratios:

Goal	Protein	Carbohydrates	Fats	Calorie Adjustment
Weight Loss	30%	40%	30%	–500 kcal/day deficit
Weight Maintenance	25%	50%	25%	No adjustment
Weight Gain / Muscle Building	35%	45%	20%	+300–500 kcal/day surplus

Table 2.4: Macronutrient Distribution by User Goal

Chapter 3: Requirements and Specifications

3.1 Introduction

This chapter presents a comprehensive specification of the requirements for the Smart Bites AI Diet Planner. It covers functional requirements (what the system must do), non-functional requirements (how well the system must perform), hardware and software requirements, use case specifications, and the data dictionary. These requirements serve as the foundation for the system design and implementation phases.

3.2 Functional Requirements

Functional requirements define the specific behaviors, functions, and capabilities that the Smart Bites system must provide to its users. These requirements are organized by user role and system module.

3.2.1 User Module Functional Requirements

Functional Requirements

- **FR-01:** System shall allow users to register an account using email, name, and password (High – Authentication)
- **FR-02:** System shall allow users to log in and log out securely (High – Authentication)
- **FR-03:** System shall allow users to create and update their health profile (age, gender, weight, height, activity level, dietary preferences, fitness goals) (High – Profile Management)
- **FR-04:** System shall calculate BMR and TDEE based on user profile using Mifflin-St Jeor equation (High – Diet Engine)
- **FR-05:** System shall generate a personalized daily meal plan (breakfast, lunch, dinner, snacks) based on calorie needs and preferences (High – Diet Engine)
- **FR-06:** System shall display meals with food items, portion sizes, calories, and macronutrients (protein, carbs, fats) (High – Diet Display)
- **FR-07:** System shall provide meal timing recommendations based on user routine (Medium – Diet Engine)

- **FR-08:** System shall suggest alternative food items based on preferences, allergies, or availability (High – Recommendation)
- **FR-09:** System shall allow users to log daily food intake (High – Food Logging)
- **FR-10:** System shall track and display daily calorie consumption vs target (High – Tracking)
- **FR-11:** System shall allow users to record weight at regular intervals (High – Weight Tracking)
- **FR-12:** System shall display weight progress charts over time (Medium – Analytics)
- **FR-13:** System shall generate and allow download of PDF diet charts (Medium – Reports)
- **FR-14:** System shall generate weekly/monthly progress reports in PDF format (Medium – Reports)
- **FR-15:** System shall allow users to reset password via email verification (High – Authentication)
- **FR-16:** System shall dynamically adjust diet plans based on weight changes and goal progress (High – Adaptation)
- **FR-17:** System shall send reminders/notifications for meals and food logging (Low – Notifications)
- **FR-18:** System shall display a dashboard summarizing intake, goals, and progress (High – Dashboard)

3.3 Non-Functional Requirements

Non-functional requirements specify the quality attributes, constraints, and performance standards that the Smart Bites system must meet:

Non-Functional Requirements

- **NFR-01:** System shall generate a diet plan within 3 seconds of user request (Performance – Response time ≤ 3 seconds)
- **NFR-02:** System shall support at least 500 concurrent users without performance degradation (Performance – 500+ concurrent users)
- **NFR-03:** All user passwords shall be stored securely using bcrypt hashing with salt (Security – bcrypt with 12+ rounds)

- **NFR-04:** All data transmission shall be secured using HTTPS/TLS encryption (Security – TLS 1.2 or higher)
- **NFR-05:** System interface shall be intuitive for users with basic computer literacy (Usability – SUS score ≥ 70)
- **NFR-06:** System shall maintain high availability during usage periods (Reliability – Uptime $\geq 99.5\%$)

3.4 Hardware and Software Requirements

3.4.1 Hardware Requirements

System Requirements

- **Processor:**
Development requires Intel Core i5 or equivalent, production server needs 2+ vCPUs (cloud-based), while client side can run on any modern processor
- **RAM:**
Development environment requires minimum 8 GB RAM, production server requires at least 4 GB, and client side requires minimum 2 GB
- **Storage:**
Development setup uses 256 GB SSD, production server requires 50 GB SSD (scalable), while client side does not require storage as it is web-based
- **Network:**
Development requires broadband internet, production server needs 100 Mbps+ dedicated connection, and client side works on any internet connection
- **Display:**
Development recommended resolution is 1920×1080, production server does not require display (headless), and client requires minimum 1024×768
- **Input Devices:**

- Development uses keyboard and mouse, production server does not require input devices, and client can use keyboard, mouse, or touch interface

3.4.2 Software Requirements

Software Requirements

- **Operating System:**
Windows 10/11 (latest version) used as the primary development platform
- **Web Server:**
XAMPP (latest version) used for local development and testing environment
- **Programming Language:**
Python 3.10+ used for backend logic and AI-based algorithms
- **Web Framework:**
FastAPI 0.100+ used for building RESTful APIs and backend services
- **Frontend Technologies:**
HTML5, CSS3, JavaScript (latest standards) used for user interface development
- **Database:**
MySQL 8.0+ or PostgreSQL 14+ used for data storage and management
- **Version Control System:**
Git (latest version) used for source code management and collaboration
- **Code Editor:**
Visual Studio Code (VS Code) used for coding, debugging, and development
- **Web Browser:**
Google Chrome or Mozilla Firefox (latest versions) used for testing and debugging
- **API Testing Tool:**
Postman used for testing and validating API endpoints
- **Documentation Tools:**
MS Word or LaTeX used for project report writing and formatting
- **PDF Generation Library:**
ReportLab or WeasyPrint used for generating diet charts and reports in PDF format

3.5 Use Case Specifications

Use cases describe the interactions between actors (users and admins) and the Smart Bites system. The following sections detail the primary use cases identified during requirements analysis.

3.5.1 Use Case: User Registration

In this use case, a new user accesses the SmartBites website and navigates to the registration page to create an account. The user provides basic details such as name, email, and password, which are validated by the system. If all inputs are valid, the system creates the account, stores it in the database, and sends a confirmation email. After successful registration, the user is redirected to complete their profile setup. In case of issues, such as an already registered email or weak password, the system displays appropriate error messages, while server-related failures prompt the user to try again later.

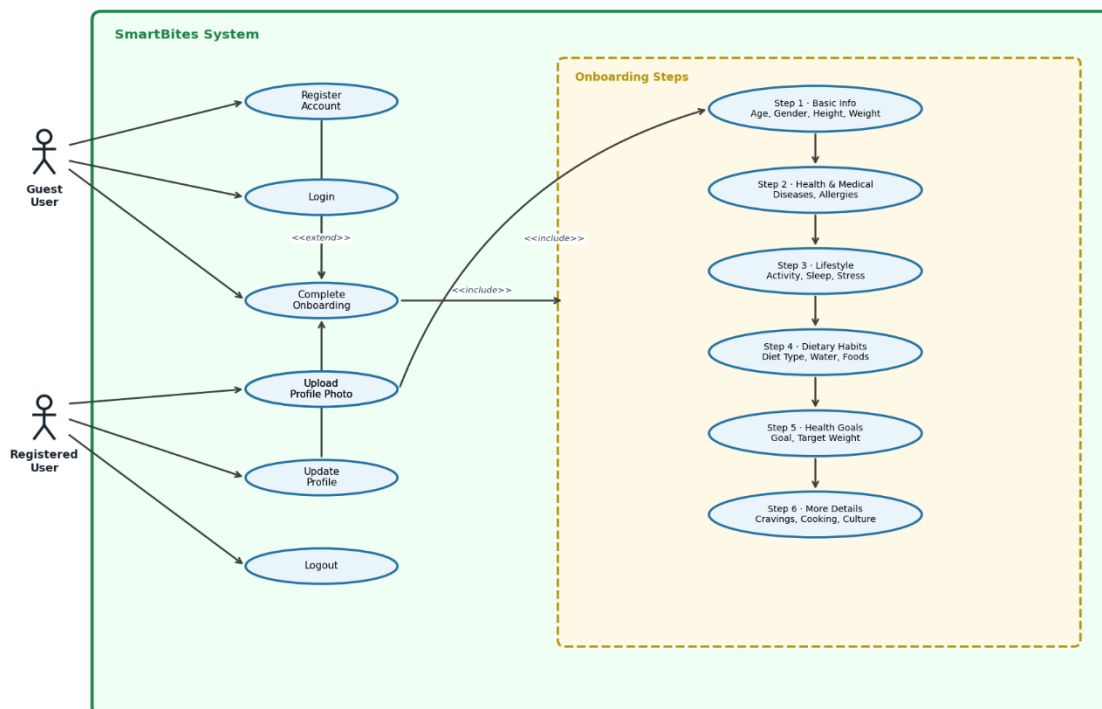


Figure 3.1: Use Case — User Registration

3.5.2 Use Case: *Generate Meal Plan*

In this use case, a registered user who is logged in and has completed their profile accesses the Diet Plan section. The system retrieves the user's profile data and calculates key metrics such as BMR and TDEE to determine an appropriate calorie target based on the user's goal. Using this information, the AI engine generates a personalized daily meal plan including breakfast, lunch, dinner, and snacks with detailed portions, calories, and macronutrient breakdown. The generated plan is then displayed to the user, who can view, save, or download it for later use.

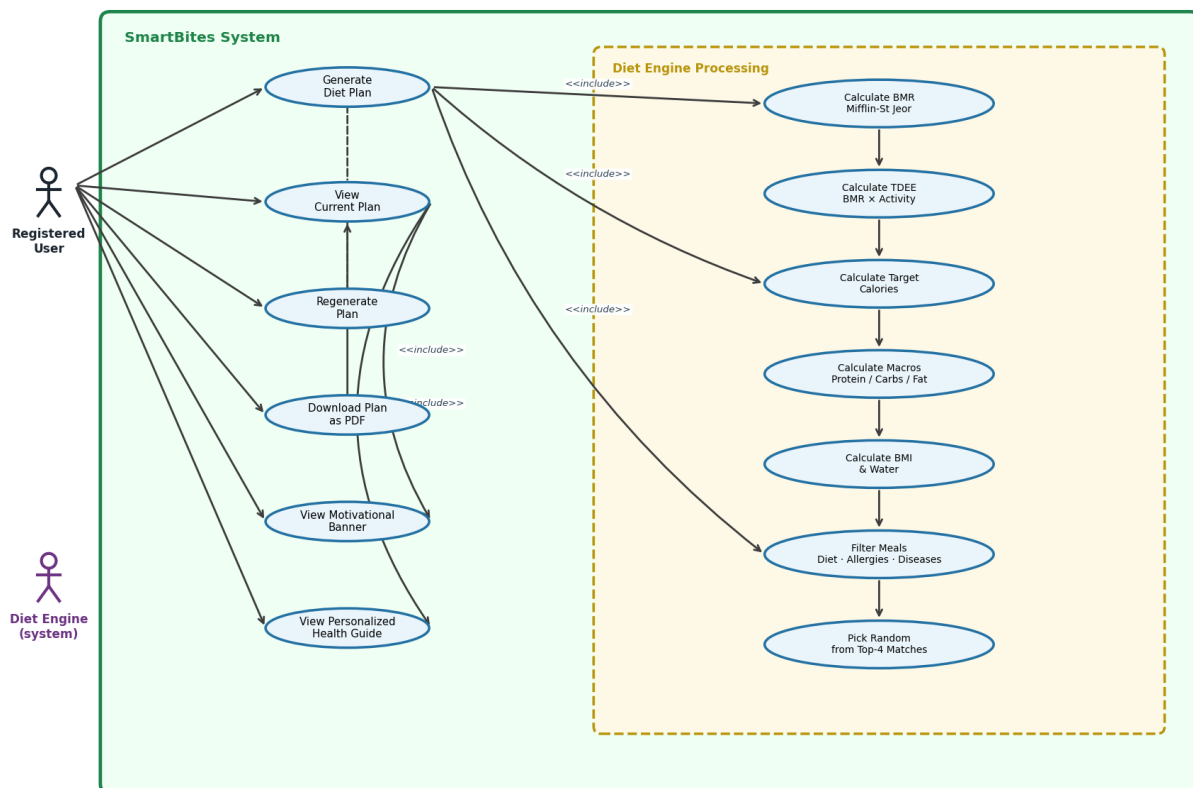


Figure 3.2: Use Case — *Generate Meal Plan*

3.5.3 Use Case: *Generate PDF Report*

In this use case, a registered user who is logged in and already has a generated diet plan navigates to the Diet Plan section and selects the “Download PDF” option. The system gathers relevant data

including the user’s profile, calorie targets, and detailed meal plan, then compiles it into a well-structured PDF report. Once generated, the file is automatically downloaded to the user’s device. If any server-side issue occurs during PDF generation, the system notifies the user with an error message and prompts them to try again

Use Case 4 • PDF Report Generation

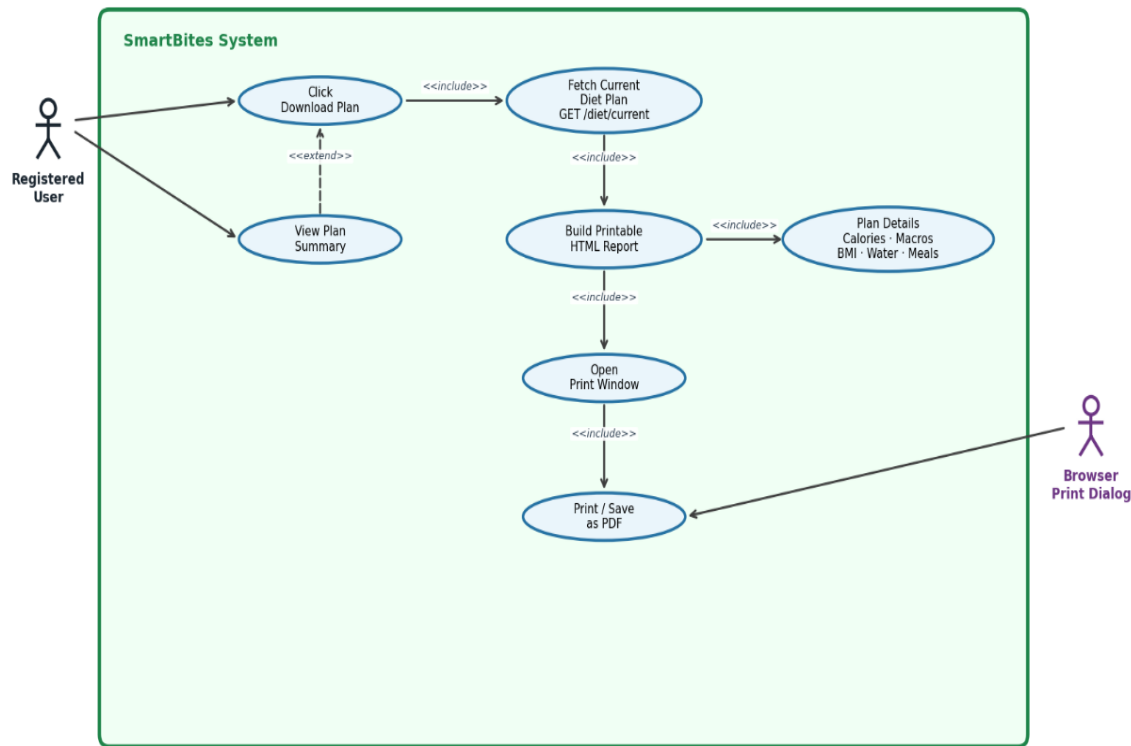


Figure 3.3: Use Case — PDF Report Generation

3.5.4 Use Case: AI Chatbot Interaction

In this use case, a registered user who is logged in accesses the AI Coach or chatbot section to seek diet and health-related guidance. The user submits a query, which is sent along with their profile context to the AI engine (Groq API — LLaMA 3.3 70B) for processing. The AI generates a personalized response based on the user’s data, and the system displays it within the chat

interface. The user can continue the interaction with follow-up questions for ongoing guidance. If the user submits an empty or irrelevant query, the system prompts them to enter a valid diet-related question, while any AI service failure or timeout results in an error message asking the user to try again later.

Use Case 5 • Chatbot Interaction (AI Coach)

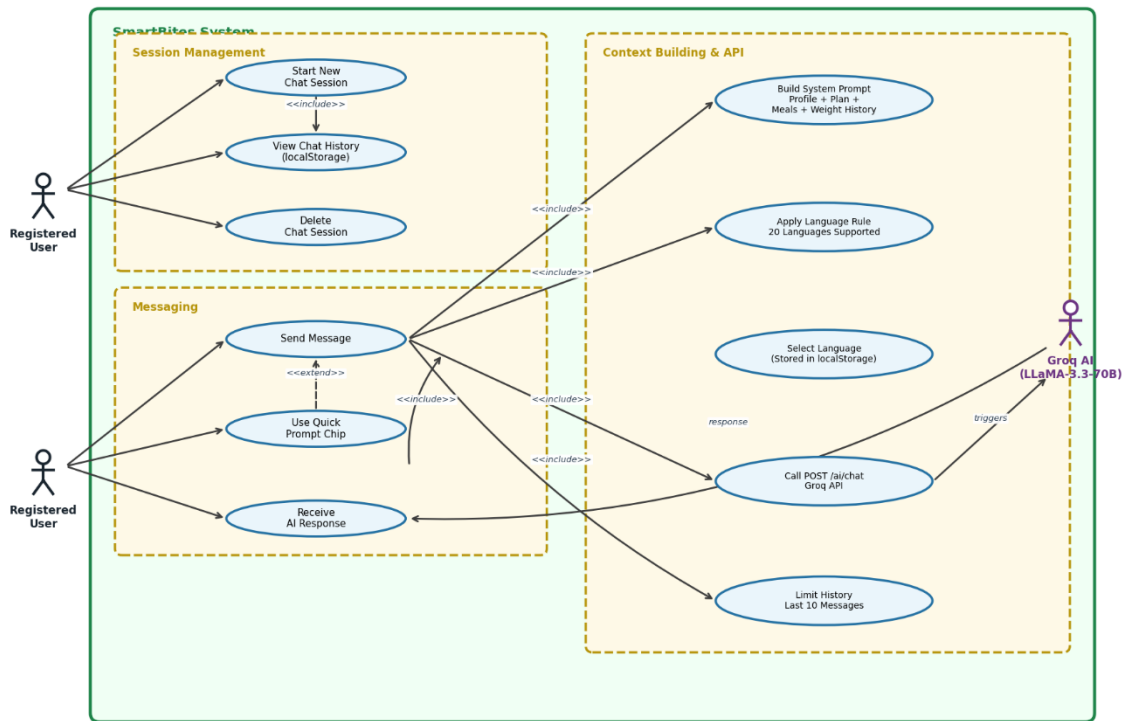


Figure 3.4: Use Case — AI Chatbot Interaction

3.5.5 Use Case: Nutritional Analysis

In this use case, a registered user who is logged in and has either a generated diet plan or logged meals accesses the Dashboard or Nutritional Analysis section. The system retrieves the user’s meal and diet data, then calculates key metrics such as total calorie intake, macronutrient distribution (protein, carbohydrates, fats), and BMI. It compares the user’s actual intake with recommended daily targets and presents a detailed nutritional summary along with progress indicators. The user can review this analysis and make adjustments to their diet preferences if needed. If no meal data is available, the system prompts the user to log meals or generate a plan,

while exceeding targets triggers warnings. Any server-related issues result in an error message asking the user to retry.

Use Case 3 · Nutritional Analysis & Visualization

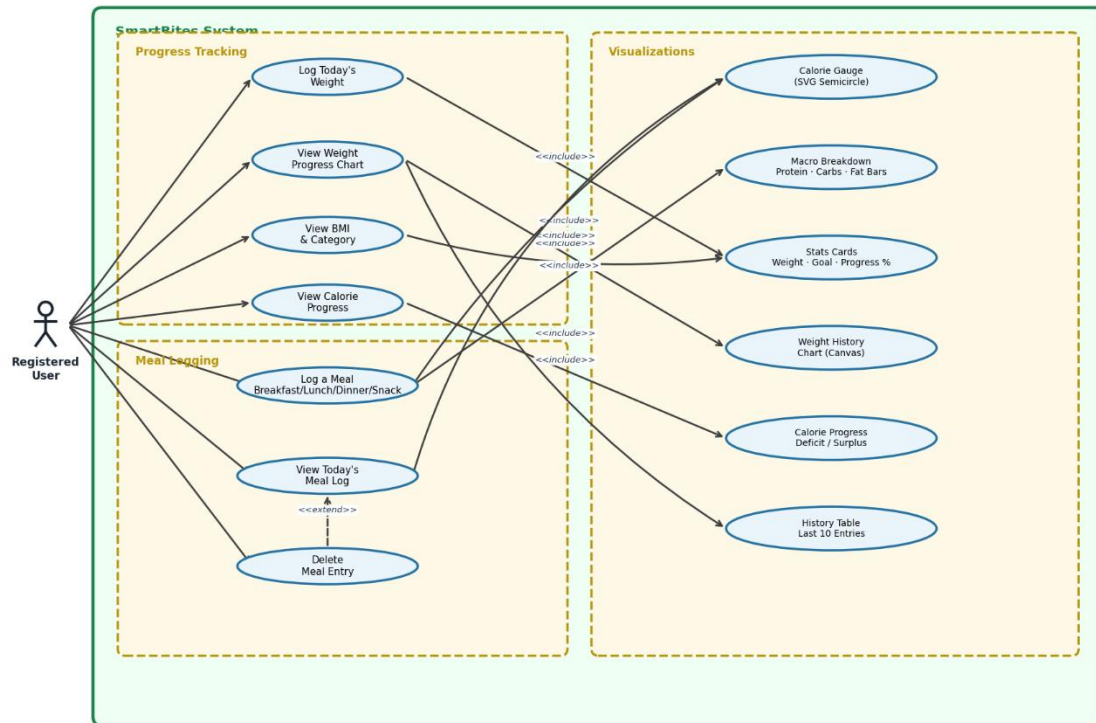


Figure 3.5: Use Case — Nutritional Analysis

Chapter 4: System Architecture

4.1 Introduction

This chapter presents the system architecture of the SmartBites AI Diet Planner. It describes the overall design approach, system components, data flow, and the interaction between different modules. The architecture is designed following a layered approach that separates the presentation layer, application logic layer, and data management layer. This separation ensures modularity, maintainability, and scalability of the system. The chapter also includes the system context diagram, high-level architecture diagram, detailed component descriptions, and the database schema design.

4.2 Architectural Design Approach

SmartBites follows a three-tier client-server architecture, which is a widely adopted design pattern for web-based applications. This architecture divides the system into three logical layers:

- **Presentation Layer (Frontend):** This is the user-facing layer built using HTML5, CSS3, and JavaScript. It provides the graphical user interface through which users interact with the system, including registration forms, profile setup pages, diet plan display, food logging interface, progress charts, and PDF download options.
- **Application Logic Layer (Backend):** This layer is developed using Python with FastAPI framework. It contains the core business logic including user authentication, BMR/TDEE calculations using the Mifflin-St Jeor equation, AI-based diet plan generation, meal recommendation algorithms, progress tracking logic, and PDF report generation. The backend exposes RESTful API endpoints that the frontend consumes.
- **Data Layer (Database):** MySQL/PostgreSQL is used to store all persistent data including user profiles, health information, food databases, generated diet plans, food intake logs, weight tracking records, and system configuration. The database is designed using normalized relational schema to minimize redundancy and ensure data integrity.

This three-tier approach ensures that changes in one layer do not directly affect the other layers, making the system easier to develop, test, and maintain independently.

4.3 System Context Diagram

The system context diagram illustrates how the SmartBites system interacts with external entities.

The primary actors in the system are:

- User: Registers, creates health profile, generates diet plans, logs food intake, tracks weight progress, downloads PDF reports, and manages account settings.
- Admin: Manages the food database, configures diet rules and AI parameters, monitors system performance, and manages user accounts.
- AI Diet Engine: An internal component that processes user data, applies the Mifflin-St Jeor equation for calorie calculation, and generates personalized meal plans using rule-based and algorithmic logic.
- Email Service: An external service used for sending account verification emails, password reset links, and optional meal reminders to users.
- Database Server: Stores all user data, food items, diet plans, intake logs, and progress records securely.

4.3.1 DFD Level 0 — Context Diagram

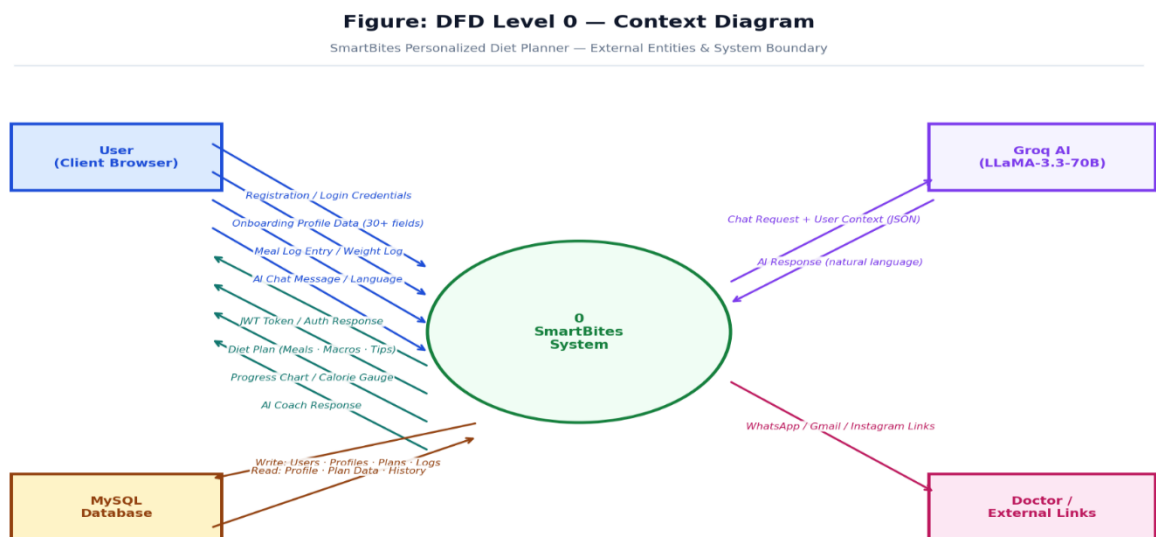


Figure 4.1: System Context Diagram

The DFD Level 0 (Context Diagram) above depicts the SmartBites system as a single central process surrounded by four external entities. The User (Client Browser) sends registration/login credentials, onboarding profile data (30+ fields), meal log entries, weight logs, and AI chat messages with language preferences to the system. In return, the system delivers JWT tokens, personalized diet plans (meals, macros, tips), progress charts with calorie gauges, and AI Coach responses. The Groq AI (LLaMA-3.3-70B) entity receives chat requests bundled with user context in JSON format and returns natural-language AI responses. The MySQL Database stores and retrieves users, profiles, plans, and logs. The Doctor / External Links entity represents outbound connections to WhatsApp, Gmail, and Instagram for doctor consultations and brand engagement.

4.4 High-Level System Architecture

The high-level architecture of SmartBites consists of the following major components that work together to deliver a seamless diet planning experience:

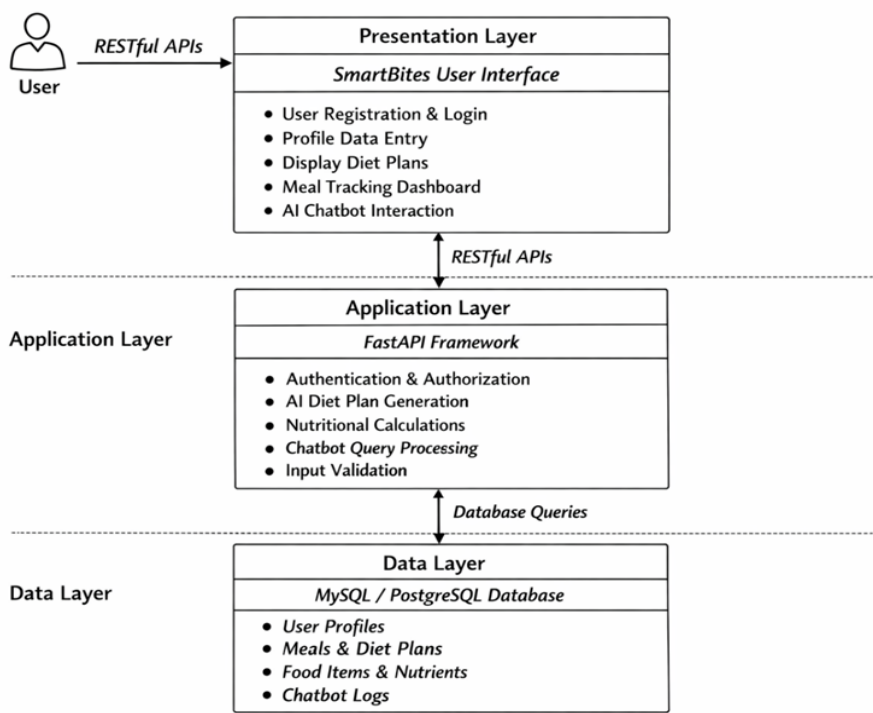


Figure 4.2: High-Level System Architecture

4.5 Detailed Module Descriptions

4.5.1 Authentication Module

This module handles user registration, login, and session management. It validates user input, checks duplicate emails, and securely stores passwords using bcrypt hashing. On login, a JWT token is generated for secure access. It also supports email-based password reset functionality.

4.5.2 Profile Management Module

This module manages user health profiles including age, gender, weight, height, activity level, dietary preferences, allergies, and fitness goals. The data is stored in the database and used by the Diet Engine. Users can update their profile anytime, and the system recalculates their diet plan accordingly.

4.5.3 Diet Engine (AI Core Module)

This is the core module of the system. It calculates BMR using the Mifflin-St Jeor equation and TDEE based on activity level. It then sets calorie targets according to user goals (loss, gain, maintenance). It distributes macronutrients and generates personalized meal plans while considering preferences and allergies. It also dynamically adjusts plans based on user progress.

4.5.4 Food Database Manager

This module manages all food items with nutritional data such as calories, protein, carbs, and fats. Foods are categorized and can be added, updated, or deleted by the admin. It serves as the main data source for meal planning and recommendations.

4.5.5 Recommendation Engine

This module suggests alternative food items with similar nutritional values. It considers user preferences, allergies, and dietary restrictions to ensure suitable replacements without affecting calorie goals.

4.5.6 Progress Tracking and Analytics Module

This module tracks user weight, calorie intake, and diet adherence. It generates charts and insights like weekly trends, average intake, and goal progress. It also helps the system adjust diet plans based on user performance.

4.5.7 Report Generator Module

This module generates downloadable PDF reports of diet plans and progress. It includes meal details, calorie breakdowns, weight charts, and progress summaries using PDF libraries like ReportLab or WeasyPrint.

4.6 Database Design

The SmartBites database is designed using a normalized relational schema to ensure data integrity, minimize redundancy, and support efficient querying. The database consists of the following core tables:

Table Name	Primary Key	Key Attributes	Purpose
users	user_id (INT, AUTO)	name, email, password_hash, role, created_at, is_verified	Stores user authentication and account data
user_profiles	profile_id (INT, AUTO)	user_id (FK), age, gender, weight, height, activity_level, dietary_preference, allergies, fitness_goal	Stores user health and preference data for diet generation
food_items	food_id (INT, AUTO)	name, category, meal_type, calories, protein, carbs, fats, fiber, serving_size, dietary_tag	Master food database with nutritional information
diet_plans	plan_id (INT, AUTO)	user_id (FK), date, total_calories, total_protein, total_carbs, total_fats, status	Stores generated daily diet plans
plan_meals	meal_id (INT, AUTO)	plan_id (FK), food_id (FK), meal_type (breakfast/lunch/dinner/snack), portion_size, calories, protein, carbs, fats	Individual meal items within a diet plan

food_logs	log_id (INT, AUTO)	user_id (FK), food_id (FK), date, meal_type, portion_size, calories_consumed	Records daily food intake by users
weight_records	record_id (INT, AUTO)	user_id (FK), date, weight, notes	Tracks user weight measurements over time
notifications	notif_id (INT, AUTO)	user_id (FK), type, message, sent_at, is_read	Stores meal reminders

Table 4.1 SmartBites Database Schema Overview

4.6.1 Entity Relationships

The key entity relationships in the SmartBites database are as follows:

- One-to-One: Each user has exactly one user profile (users → user_profiles).
- One-to-Many: Each user can have multiple diet plans (users → diet_plans), multiple food logs (users → food_logs), and multiple weight records (users → weight_records).
- One-to-Many: Each diet plan contains multiple meal items (diet_plans → plan_meals).
- Many-to-One: Each plan meal references one food item from the food database (plan_meals → food_items).
- One-to-Many: Each user can receive multiple notifications (users → notifications).

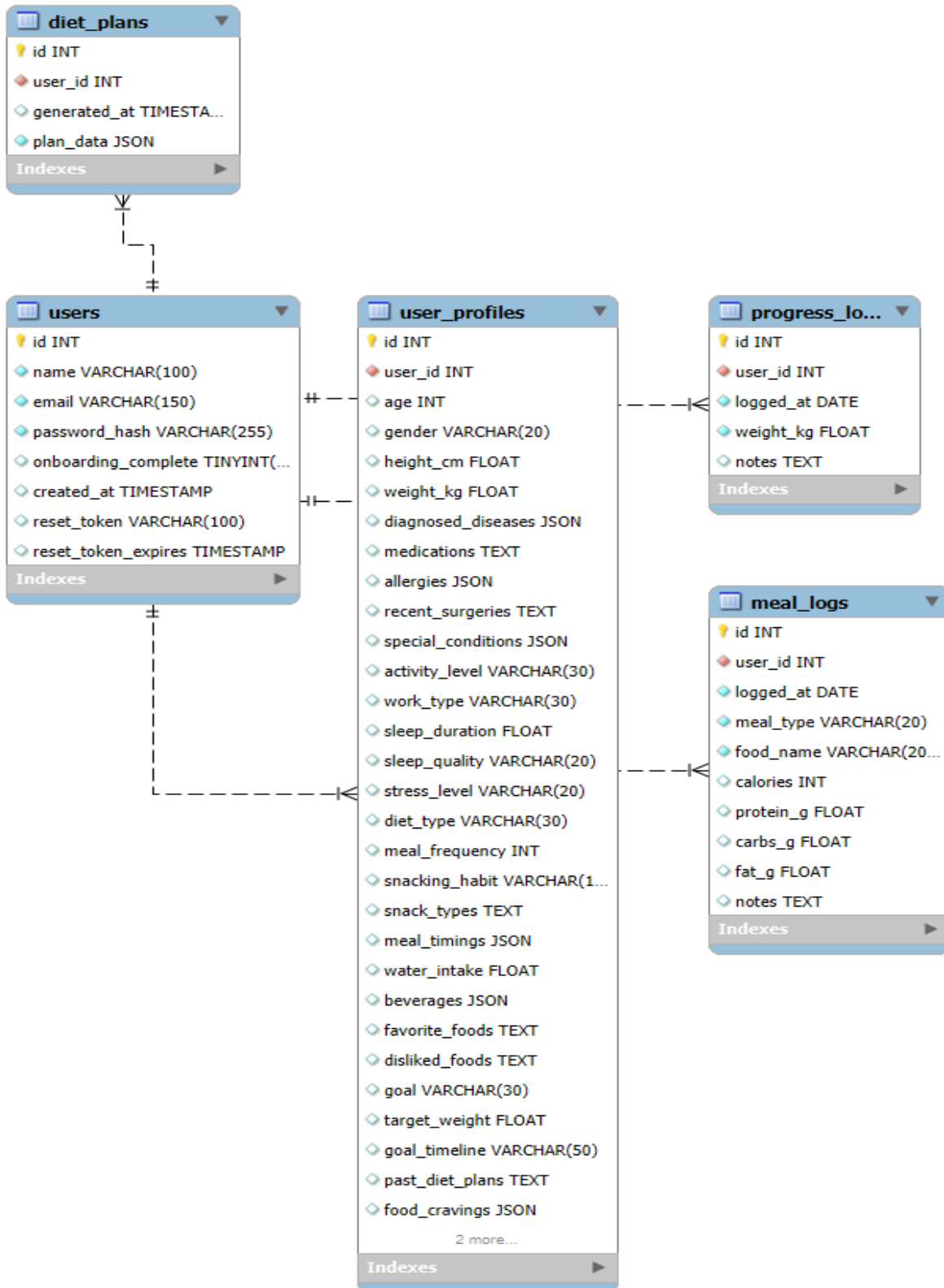


Figure 4.3: ER Diagram

4.7 Data Flow Description

4.7.1 DFD Level 1 — Detailed Data Flow Diagram

- User interacts with the web interface (forms, buttons) in the presentation layer.
- Frontend sends HTTP requests (GET, POST, PUT, DELETE) to FastAPI backend via REST APIs.
- API layer validates JWT token and routes request to the appropriate module.
- Module processes request (e.g., Diet Engine calculates BMR/TDEE, fetches food data, generates meal plan).
- System interacts with database using ORM for reading/writing data.
- Backend returns processed data in JSON format.
- Frontend displays results as diet plans, charts, or messages in UI.

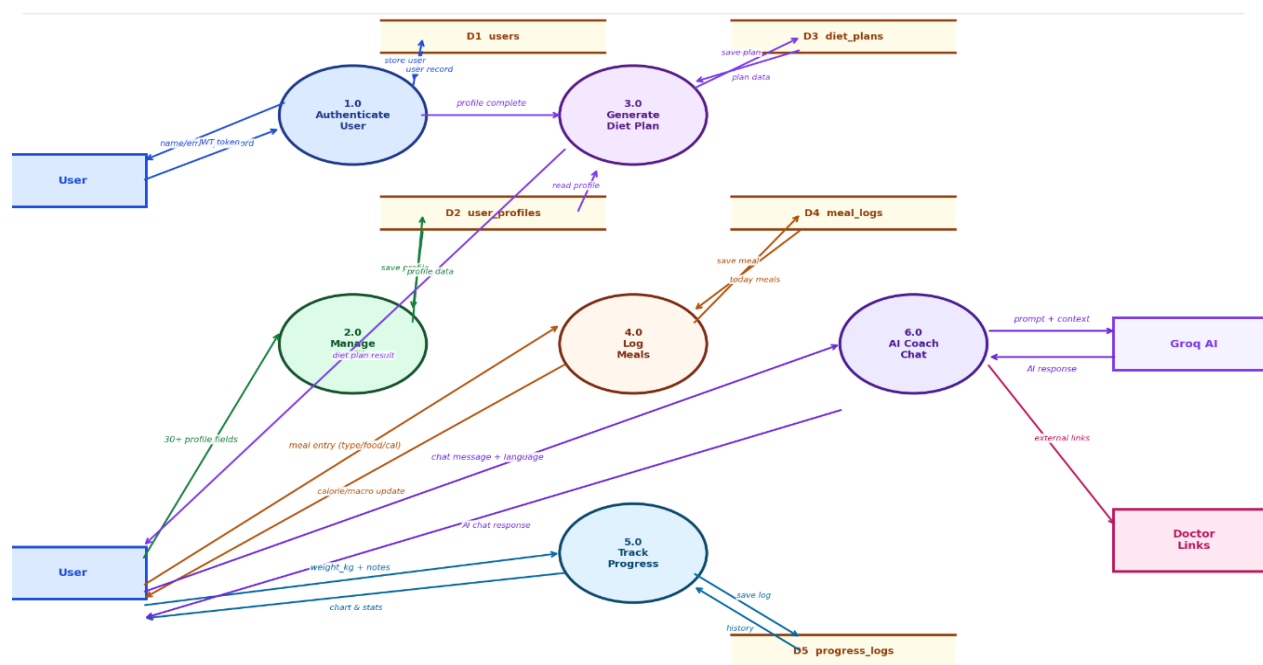


Figure 4.4: Data Flow Diagram (DFD)

4.8 Sequence diagram

The Sequence diagram illustrates interactions between User, Frontend, FastAPI Backend, Diet Engine, MySQL DB, and Groq AI across key workflows. During registration and onboarding, user data is submitted, stored in the database, and authenticated using JWT. In the diet generation

phase, the backend processes profile data, calculates BMR/TDEE, generates a personalized plan, and stores it for dashboard display. Meal logging updates calorie tracking in real time, while the AI chatbot uses user context and history to generate personalized responses via the Groq API.

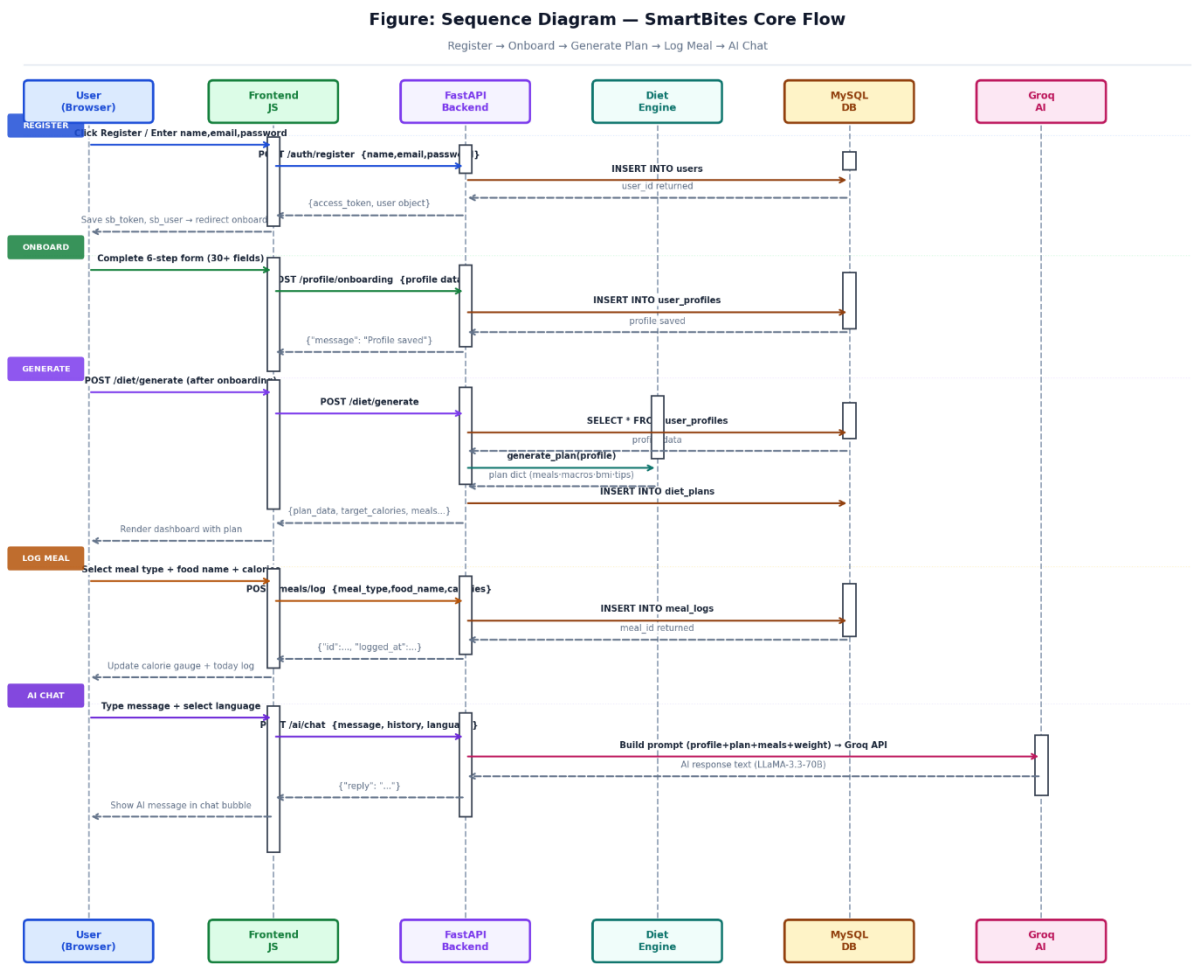


Figure 4.5: Sequence Diagram — Core Flow

4.8 Security Architecture

Security is a critical consideration in the SmartBites system as it handles sensitive personal and health data. The following security measures are implemented across all layers of the architecture:

- **Authentication Security:** User passwords are hashed using bcrypt with a minimum of 12 salt rounds before storage. JWT tokens are used for stateless session management with configurable expiration times.
- **Data Transmission Security:** All communication between the client and server is encrypted using HTTPS with TLS 1.2 or higher, preventing man-in-the-middle attacks and data interception.
- **Input Validation:** All user inputs are validated on both the frontend (client-side) and backend (server-side) to prevent SQL injection, cross-site scripting (XSS), and other injection attacks.
- **Access Control:** Role-based access control (RBAC) ensures that users can only access their own data, while admins have elevated privileges for system management. API endpoints are protected with authentication middleware.
- **Database Security:** Sensitive data fields are encrypted at rest. Database connections use parameterized queries through ORM to prevent SQL injection. Regular database backups ensure data recovery in case of failures.

4.8.1 Activity Diagram — SmartBites User Activities

The following activity diagram illustrates the complete user journey through the SmartBites application, capturing all decision points, parallel activities, and system interactions from initial app launch to logout. This UML-based activity diagram models the behavioral flow of the system and provides a visual representation of how users navigate through various features and modules.

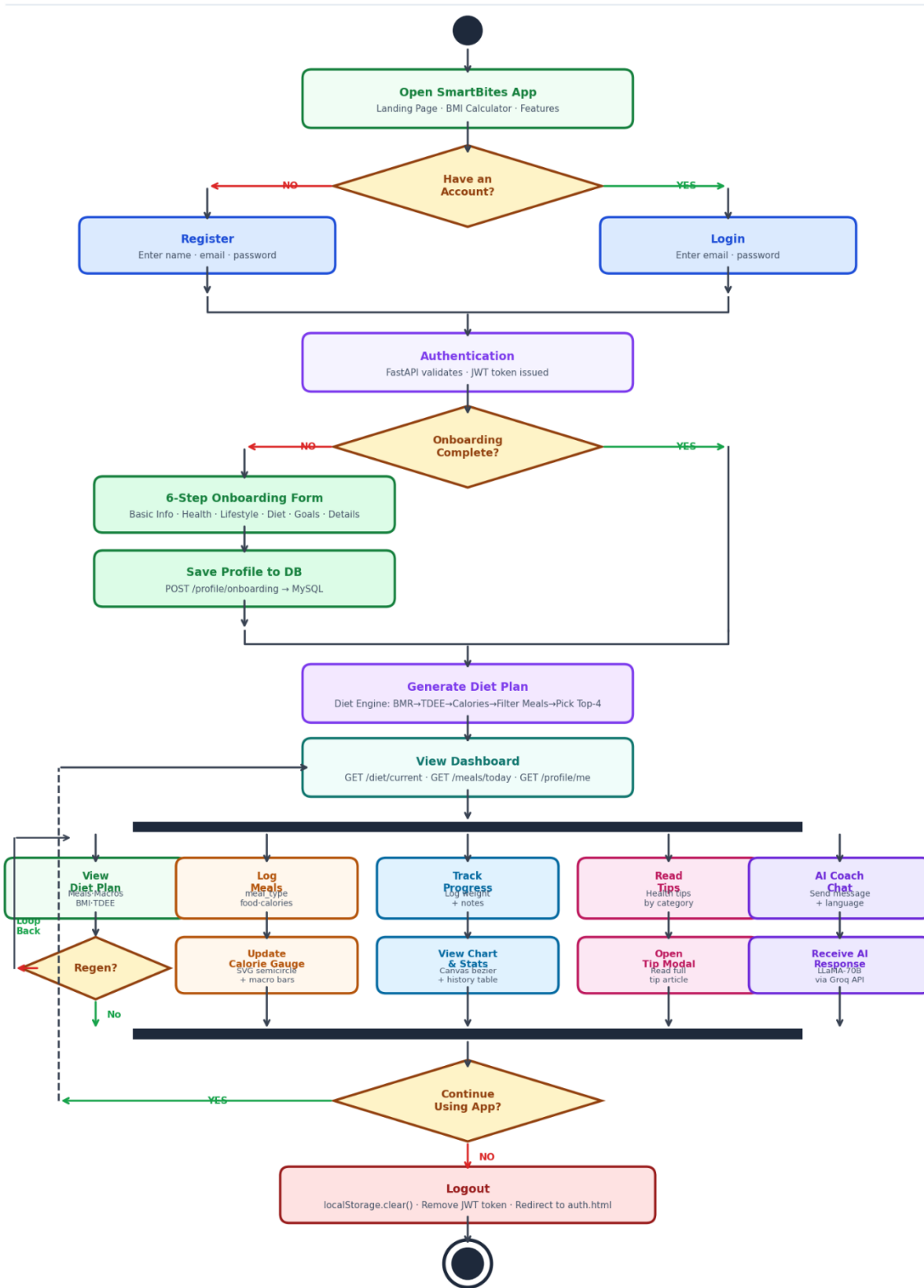


Figure 4.6: Activity Diagram

4.8.2 UI/UX Design

The UI/UX design of the SmartBites application prioritizes simplicity, accessibility, and a seamless user experience across all devices. The interface is designed to guide users intuitively through every step — from registration and onboarding to daily meal tracking and AI-powered coaching. This section outlines the core design principles, screen flow architecture, and key interface components that shape the SmartBites user experience.

Design Principles

The SmartBites UI/UX is built on the following core design principles that ensure a consistent and engaging experience:

- **User-Centered Design** — Every screen and interaction is designed from the user's perspective. The 6-step onboarding form collects health data progressively to avoid overwhelming new users, while returning users bypass onboarding entirely for a frictionless experience.
- **Visual Hierarchy and Clarity** — Key information such as calorie targets, BMI values, and diet plan summaries is presented with clear typographic hierarchy. Bold headings, color-coded status indicators, and card-based layouts guide the user's attention to actionable data.
- **Responsive and Mobile-First Layout** — The interface is built using responsive HTML5 and CSS3, ensuring optimal rendering on desktops, tablets, and mobile devices. All interactive elements (buttons, forms, charts) adapt to varying screen sizes without loss of functionality.
- **Minimalist Navigation** — The application uses a streamlined navigation structure: Landing Page → Authentication → Onboarding → Dashboard. Within the dashboard, a single-page interface with tabbed or card-based modules (Diet Plan, Meal Log, Progress, Tips, AI Coach) eliminates unnecessary page reloads.
- **Accessibility and Readability** — The color palette uses high-contrast combinations for text readability. Form inputs include validation feedback (green for valid, red for errors). Button labels use clear action verbs ("Get Started", "Generate Plan", "Log Meal") to reduce cognitive load.
- **Feedback and State Communication** — Every user action triggers immediate visual feedback: loading spinners during API calls, success toasts after saving data, and error modals with

actionable messages when operations fail. The calorie gauge and macro bars update in real-time as meals are logged.

Screen Flow Architecture

The UI/UX flow diagram above maps each screen transition in the application. The following describes the visual design and user interaction at each major screen:

Landing Page

The landing page serves as the first point of contact. It features a hero section with the SmartBites branding, a quick BMI calculator for immediate user engagement, and a feature overview section highlighting the app's core capabilities. Clear call-to-action buttons ("Get Started" and "Sign Up") direct users toward registration or login.

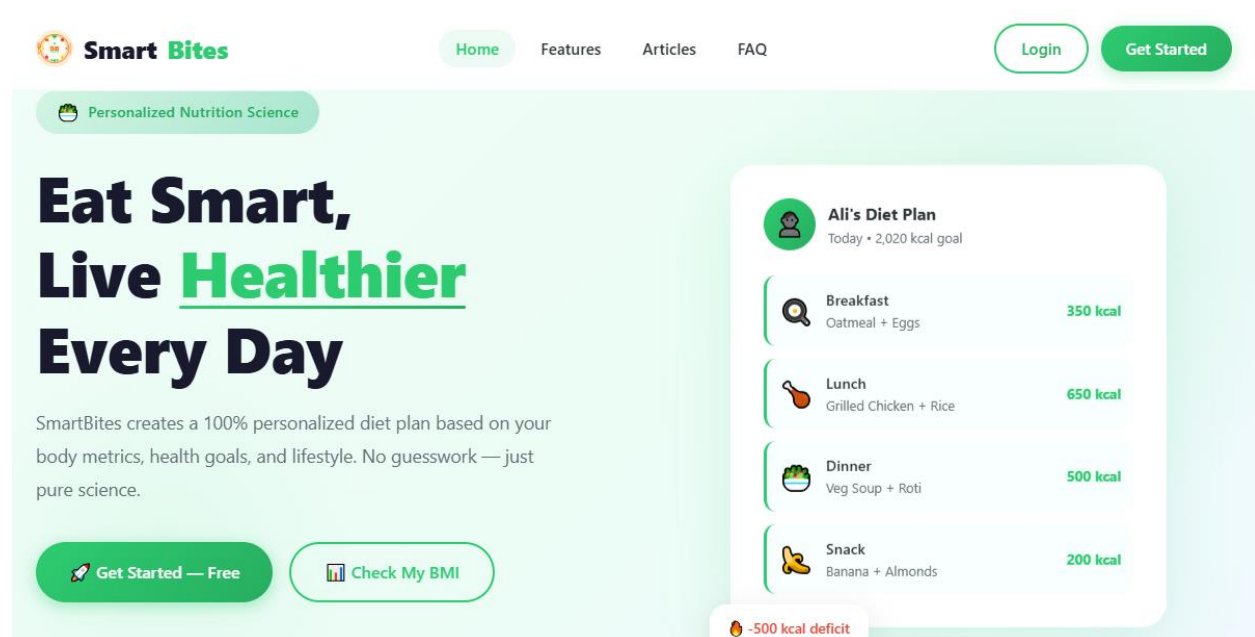


Figure 4.7: Landing Page — Hero Section

Authentication Screens (Login / Register)

The authentication interface uses a clean, centered card layout with clearly labeled input fields. The registration form collects name, email, and password with real-time validation indicators. The login form provides email and password fields with a "Forgot Password" link. Both forms share a consistent visual style and toggle seamlessly between login and registration modes.

6-Step Onboarding Form

The onboarding process uses a multi-step wizard pattern with a progress indicator at the top showing the current step (1 of 6). Each step focuses on a single category — Basic Info, Health, Lifestyle, Diet, Goals, and Details — preventing information overload. Navigation buttons ("Next" and "Back") allow users to move between steps, and all inputs are validated before progression. The wizard design reduces form abandonment by breaking a long questionnaire into manageable segments.

Main Dashboard

The dashboard is the central hub of the SmartBites application. It uses a card-based layout to present five core modules simultaneously: Diet Plan, Meal Log, Progress Tracker, Health Tips, and AI Coach. Each card displays a summary view with key metrics (e.g., today's calorie target, meals logged, weight trend) and expands into a detailed view on interaction. The dashboard aggregates data from three API endpoints (GET /diet/current, GET /meals/today, GET /profile/me) to provide a unified real-time overview of the user's health journey.

Diet Plan View

The diet plan screen displays the personalized meal plan generated by the Diet Engine. It presents BMI and TDEE values at the top, followed by a structured meal layout organized by time slot (breakfast, lunch, dinner, snacks). Each meal card shows the food item name, portion size, and calorie count. A prominent "Regenerate Plan" button allows users to request a new plan with randomized meal selections while maintaining the same calorie targets.

Calorie Gauge and Meal Logging

The meal logging interface provides a searchable food selector where users can log individual meals with calorie and food category information. Upon logging, the Calorie Gauge — a circular progress indicator — updates in real-time to show the percentage of daily calorie intake consumed. Macro nutrient bars (protein, carbohydrates, fats) are displayed below the gauge, giving users an at-a-glance view of their nutritional balance throughout the day.

Progress Charts and Statistics

The progress tracking module uses Canvas-based visualizations to render interactive charts including a calorie intake trend line, macronutrient distribution pie chart, and weight progress graph. A history table below the charts lists daily entries with date, weight, calories consumed, and personal notes. Users can filter data by time range (daily, weekly, monthly) to analyze trends over different periods.

Health Tips Interface

The health tips module presents curated articles organized by category (nutrition, exercise, mental health, hydration). Each tip is displayed as a compact card with a title and category tag. Clicking a card opens a Tip Modal — a full-screen overlay that presents the complete article with formatted text, ensuring a distraction-free reading experience.

AI Coach Chat Interface

The AI Coach module features a conversational chat interface inspired by modern messaging apps. Users type diet or health-related questions in a text input field with an optional language selector. The system sends the query with user profile context to the Groq API (LLaMA 3.3 70B), and the AI-generated response appears in a styled message bubble. The chat supports multi-turn conversations, allowing users to ask follow-up questions within the same session.

Chapter 5: Implementation

5.1 Introduction

This chapter presents the detailed implementation of the SmartBites AI Diet Planner. It covers the development environment setup, the technology stack used, and the step-by-step implementation of each system module. SmartBites is built as a web-based application specifically designed for Pakistani and Indian users, featuring a hand-curated local food database, culturally aware dietary filters (halal, vegetarian, Jain, Hindu, kosher), and multi-language AI coaching. The system uses a 6-step onboarding form to collect comprehensive user data, generates personalized meal plans using the Mifflin-St Jeor equation combined with goal-aware meal selection, and integrates an AI nutritionist chatbot powered by Groq API with LLaMA 3.3 70B model. This chapter explains how each feature was translated from the architectural design into working code.

5.2 Development Environment Setup

The development environment for SmartBites was configured on Windows operating system using the following setup. Python 3.11 was installed as the primary backend runtime. A virtual environment was created using venv to isolate project dependencies. The FastAPI framework was installed along with Uvicorn as the ASGI server for running the backend. MySQL was used as the database server, managed through XAMPP for local development and testing. SQLAlchemy ORM was configured for database interaction. The frontend was developed using vanilla HTML5, CSS3, and JavaScript without any framework dependency, ensuring lightweight and fast page loads. VS Code was used as the primary code editor with Python and JavaScript extensions.

5.3 Technology Stack

The following table summarizes the complete technology stack used in the development of SmartBites:

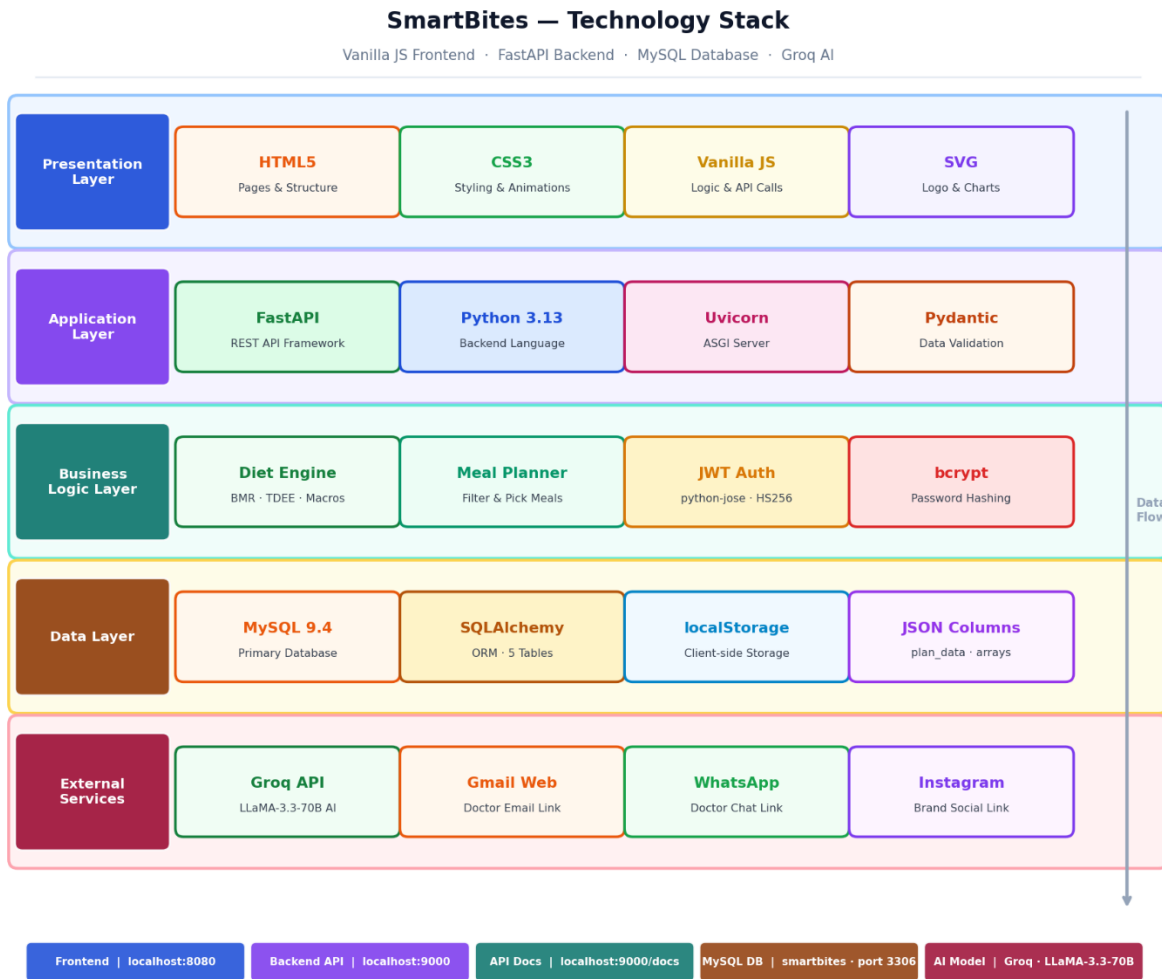


Figure 5.1: Technology Stack Architecture

5.4 Frontend Implementation

The frontend of SmartBites is built using vanilla HTML5, CSS3, and JavaScript without relying on any frontend framework such as React or Angular. This decision was made to keep the application lightweight, reduce load times, and avoid unnecessary complexity for a project of this scope. The frontend communicates with the FastAPI backend through asynchronous `fetch()` API calls, sending and receiving JSON data over RESTful endpoints.

5.4.1 Page Structure and Navigation

The application follows a multi-page architecture with the following key pages:

- **Landing Page:** The entry point of the application featuring a hero section with the SmartBites tagline, feature highlights, and call-to-action buttons for registration and login.
- **Authentication Pages (Login/Register):** Separate forms for user login and registration with client-side validation. Registration collects name, email, and password. Login accepts email and password, and on success stores the JWT token in localStorage.
- **Onboarding Pages (6 Steps):** A multi-step guided form that collects the user's complete health profile. Each step is displayed as a separate card with a progress indicator at the top.
- **Dashboard:** The main user interface after onboarding, divided into six sections — Diet Plan, Progress, Log Meals, Health Tips, Profile, and Chat with AI Doctor.

5.4.2 Key Frontend Features

The frontend implements the following key features:

- **Responsive Design:** CSS media queries ensure the interface adapts to desktop, tablet, and mobile screen sizes. Flexbox and CSS Grid are used for layout management.
- **Dynamic Content Rendering:** JavaScript dynamically populates diet plan cards, progress charts, and meal log entries based on API responses without page reloads.
- **Chart.js Integration:** Interactive line charts and bar charts display weight progress over time, daily calorie intake vs. target, and macronutrient distribution breakdowns.
- **Form Validation:** Client-side validation on all forms ensures that required fields are filled, email format is correct, password meets minimum length, and numeric fields (age, weight, height) contain valid values before submission.
- **JWT Token Management:** The authentication token is stored in localStorage after login and automatically included in the Authorization header of every subsequent API request. Token expiry is handled by redirecting to the login page.
- **PDF Download:** A download button on the diet plan page triggers an API call that returns the diet chart as a styled PDF file, which the browser downloads automatically.

BMI Calculator
Enter your details to get your result

AGE
25 yrs

HEIGHT
175 cm ft / in

WEIGHT
70 kg lbs

Calculate My BMI →

Figure 5.2: BMI Calculator on Landing Page

5.5 Backend Implementation

The backend of SmartBites is developed using Python 3.11 with the FastAPI framework. FastAPI was chosen for its high performance, automatic API documentation (Swagger UI), built-in data validation via Pydantic models, and native support for asynchronous programming. Uvicorn serves as the ASGI server. The backend handles all business logic including user authentication, profile management, diet plan generation, meal logging, progress tracking, AI chatbot integration, and PDF report generation.

5.5.1 RESTful API Endpoints

The backend exposes the following RESTful API endpoints organized by module:

Endpoint	Method	Module	Description
/api/auth/register	POST	Authentication	Register new user with email, name, password

/api/auth/login	POST	Authentication	Authenticate user, return JWT token (7-day expiry)
/api/auth/reset-password	POST	Authentication	Send password reset link via email
/api/profile/create	POST	Profile	Save 6-step onboarding data (health profile)
/api/profile/update	PUT	Profile	Update existing user profile and preferences
/api/profile/get	GET	Profile	Retrieve current user profile data
/api/diet/generate	POST	Diet Engine	Generate personalized daily meal plan
/api/diet/regenerate	POST	Diet Engine	Regenerate plan with different meal selections
/api/diet/current	GET	Diet Engine	Get current active diet plan
/api/diet/download-pdf	GET	Report	Download diet plan as styled PDF

/api/meals/log	POST	Meal Logging	Log food intake with calorie data
/api/meals/today	GET	Meal Logging	Get today's logged meals and calorie gauge
/api/progress/log-weight	POST	Progress	Record daily weight measurement
/api/progress/chart	GET	Progress	Get weight trend data for chart display
/api/chat/send	POST	AI Coach	Send message to AI nutritionist chatbot
/api/chat/history	GET	AI Coach	Retrieve chat conversation history

Table 5.1: SmartBites RESTful API Endpoints

```

1 app = FastAPI(title='SmartBites API', version='1.0.0')
2
3 # — Route Registration —
4 app.include_router(auth.router)      # /auth
5 app.include_router(profile.router)   # /profile
6 app.include_router(diet.router)      # /diet
7 app.include_router(progress.router)  # /progress
8 app.include_router(meals.router)     # /meals
9 app.include_router(ai.router)        # /ai

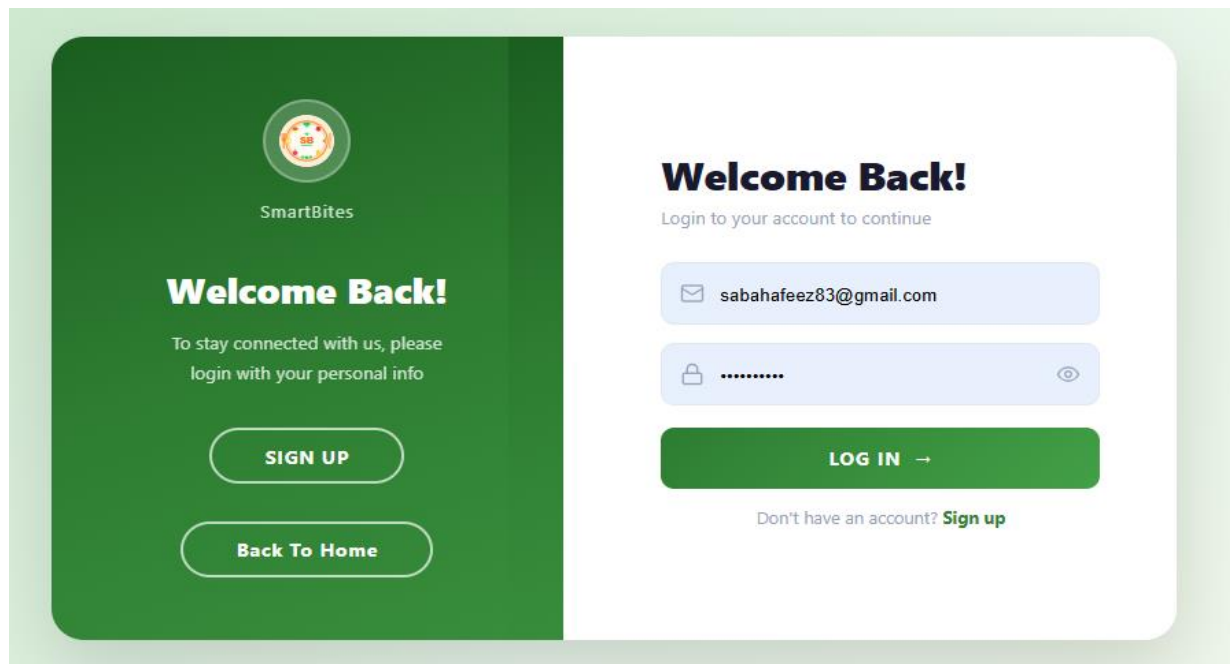
```

Figure 5.3: FastAPI Route Registration (main.py)

5.5.2 Authentication Implementation

User authentication in SmartBites is implemented using JSON Web Tokens (JWT) with bcrypt password hashing. During registration, the user's password is hashed using bcrypt with 12 salt rounds and stored in the database — the plain-text password is never saved. Upon login, the system verifies the email and password hash, and on success generates a JWT token with a 7-day expiration period. This token is returned to the frontend and stored in localStorage. Every subsequent API request includes this token in the Authorization header as a Bearer token. The backend middleware validates the token before processing any protected endpoint. If the token is expired or invalid, the request is rejected with a 401 Unauthorized response. Password reset is handled through a time-limited token sent to the user's registered email address.

Figure 5.4: Login Page — Authentication



5.6 Database Implementation

SmartBites uses MySQL as the primary relational database management system, with SQLAlchemy as the Object-Relational Mapping (ORM) layer. SQLAlchemy allows the application

to define database models as Python classes, making database operations more readable and maintainable. XAMPP is used to run the MySQL server during local development and testing.

5.6.1 Key Database Models

The following SQLAlchemy models define the core database schema. Each model maps to a MySQL table:

User Table:

Contains id, name, email, password_hash, role, verification status, and creation date. One user can have one profile and multiple diet plans, weight records, and food logs.

UserProfile Table:

Stores health-related data such as age, gender, weight, height, activity level, diet type, allergies, diseases, medications, sleep, stress level, meal frequency, food preferences, cultural restrictions, and health goals. Linked to one user.

FoodItem Table:

Stores food details including name, category, meal type, nutritional values (calories, protein, carbs, fats, fiber), serving size, dietary tags, allergen flags, and cuisine type. Used in diet plans and food logs.

DietPlan Table:

Stores generated diet plans with user ID, date, total and target calories, macronutrients, goal type, and status. Each user can have multiple diet plans.

PlanMeal Table:

Stores individual meals in a diet plan including meal slot (breakfast, lunch, dinner, etc.), portion size, calories, and macronutrients. Linked to both diet plan and food items.

FoodLog Table:

Tracks user food intake with food ID, date, meal type, portion size, and calories consumed. Linked to user and food items.

WeightRecord Table:

Stores user weight entries with date, weight value, and optional notes for progress tracking.

ChatMessage Table:

Stores chatbot conversations including user messages, AI responses, selected language, and timestamp. Linked to user.

5.6.2 Food Database

A key feature of SmartBites is its curated food database focused on Pakistani and Indian cuisine. Unlike generic diet apps, it includes detailed nutritional data for local foods such as daal, chapati, biryani, nihari, haleem, paratha, lassi, and raita. Each food item is categorized by cuisine type, meal suitability (breakfast, lunch, dinner, snack), dietary type (vegetarian, non-vegetarian, vegan, egg-free), and allergen information. It also supports cultural and religious filters such as halal, Jain, kosher, and Hindu dietary rules. Additionally, foods are tagged according to fitness goals, including low-calorie options for weight loss, high-protein meals for muscle gain, and calorie-dense foods for weight gain, making the system highly personalized and region-specific.

5.7 AI Integration — Groq API with LLaMA 3.3 70B

SmartBites integrates an AI-powered nutritionist chatbot using the Groq API with the LLaMA 3.3 70B large language model. This feature allows users to have real-time conversations with a virtual AI doctor about their diet, nutrition questions, health concerns, and meal planning advice. The chatbot is accessible from the dashboard's "Chat with Dr" section.

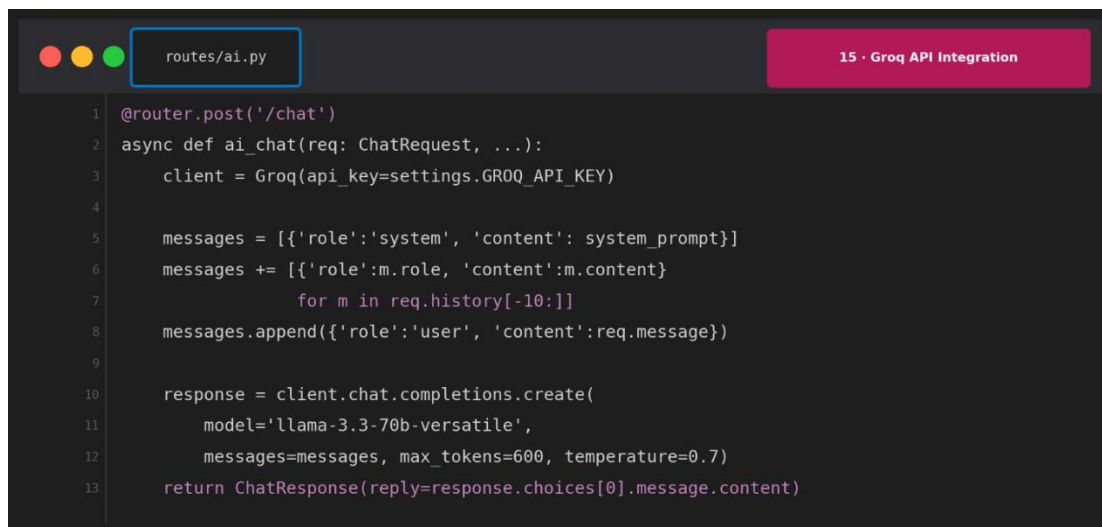
5.7.1 How the AI Coach Works

The AI chatbot allows users to ask nutrition-related questions in the chat interface and select from 20 supported languages. The frontend sends the message to the `/api/chat/send` endpoint along with the user's JWT token and selected language.

The backend creates a system prompt using the user's health profile (age, weight, height, goals, allergies, and diseases) to provide context-aware responses. The request is then sent to the Groq

API using the LLaMA 3.3 70B model, which generates a response in the selected language as a professional nutritionist.

The response is stored in the database and returned to the frontend in real time for display in the chat interface. Users can continue the conversation through follow-up queries. The system supports multiple languages including Urdu, English, Hindi, Arabic, Punjabi, Sindhi, Bengali, Turkish, and Persian, making it accessible to a wide South Asian audience.



```
1 @router.post('/chat')
2 async def ai_chat(req: ChatRequest, ...):
3     client = Groq(api_key=settings.GROQ_API_KEY)
4
5     messages = [{'role': 'system', 'content': system_prompt}]
6     messages += [{'role': m.role, 'content': m.content}
7                 for m in req.history[-10:]]
8     messages.append({'role': 'user', 'content': req.message})
9
10    response = client.chat.completions.create(
11        model='llama-3.3-70b-versatile',
12        messages=messages, max_tokens=600, temperature=0.7)
13    return ChatResponse(reply=response.choices[0].message.content)
```

Figure 5.5: AI Coach — Groq API Integration (routes/ai.py)



Figure 5.6: AI Coach Chat Interface

5.8 User Onboarding — 6-Step Form

After registration, SmartBites guides the user through a comprehensive 6-step onboarding form. This form collects all the data necessary for generating an accurate and personalized diet plan. Each step is presented as a separate card with a visual progress indicator showing the completion status. Users can navigate back and forth between steps before final submission. The following table describes the data collected in each step:

Step 1 – Basic Information:

Collects age, height (cm), current weight (kg), and gender (male, female, other)

Step 2 – Health Conditions:

Collects diagnosed diseases (diabetes, hypertension, PCOS, thyroid, cholesterol, etc.), allergies (15 common allergens like gluten, dairy, nuts, soy, shellfish, eggs), and current medications

Step 3 – Lifestyle:

Collects daily activity level (sedentary to extra active), average sleep hours per night, and stress level (low, medium, high)

Step 4 – Dietary Preferences:

Collects diet type (vegetarian, non-vegetarian, vegan, pescatarian, eggetarian), meal frequency preferences, and cultural/religious restrictions (halal, Jain, kosher, Hindu, none)

Step 5 – Food Preferences:

Collects liked foods, disliked foods, and preferred cuisines (Pakistani, Indian, International, mixed)

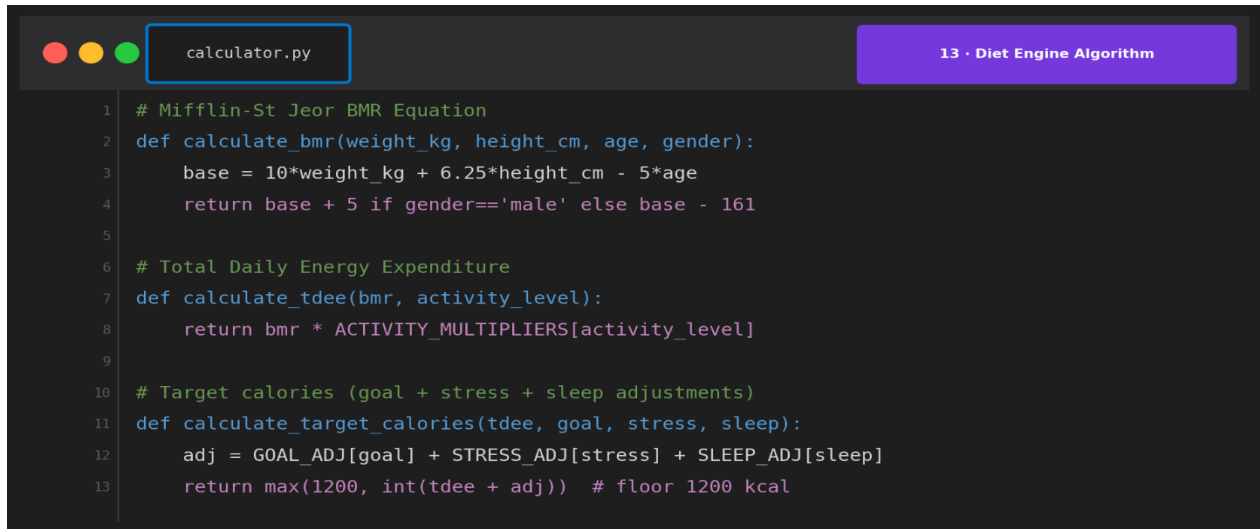
Step 6 – Health Goals:

Collects primary goal (lose weight, gain weight, build muscle, maintain weight, eat healthier), optional target weight, and timeline preference

Upon completing all six steps and clicking Submit, the profile data is validated on both client and server side, then stored in the `user_profiles` table. The Diet Engine is automatically triggered to generate the user's first personalized meal plan based on the submitted profile.

5.9 Diet Plan Generation Algorithm

The diet plan generation is the core intelligence of SmartBites. When a user requests a new diet plan (either after onboarding or through the Regenerate button), the Diet Engine executes the following algorithm:



```
1 # Mifflin-St Jeor BMR Equation
2 def calculate_bmr(weight_kg, height_cm, age, gender):
3     base = 10*weight_kg + 6.25*height_cm - 5*age
4     return base + 5 if gender=='male' else base - 161
5
6 # Total Daily Energy Expenditure
7 def calculate_tdee(bmr, activity_level):
8     return bmr * ACTIVITY_MULTIPLIERS[activity_level]
9
10 # Target calories (goal + stress + sleep adjustments)
11 def calculate_target_calories(tdee, goal, stress, sleep):
12     adj = GOAL_ADJ[goal] + STRESS_ADJ[stress] + SLEEP_ADJ[sleep]
13     return max(1200, int(tdee + adj)) # floor 1200 kcal
```

Figure 5.7: Diet Engine Algorithm (calculator.py)

5.9.1 Step 1 — BMR Calculation (Mifflin-St Jeor Equation)

The Basal Metabolic Rate (BMR) is calculated using the Mifflin-St Jeor equation, which is considered the most accurate formula for estimating resting energy expenditure:

- For Males: $BMR = 10 \times \text{weight(kg)} + 6.25 \times \text{height(cm)} - 5 \times \text{age(years)} + 5$
- For Females: $BMR = 10 \times \text{weight(kg)} + 6.25 \times \text{height(cm)} - 5 \times \text{age(years)} - 161$

5.9.2 Step 2 — TDEE Calculation

The Total Daily Energy Expenditure (TDEE) is calculated by multiplying the BMR with an activity factor based on the user's reported activity level:

Activity Level	Multiplier	Description
Sedentary	1.2	Little or no exercise, desk job
Lightly Active	1.375	Light exercise 1-3 days/week
Moderately Active	1.55	Moderate exercise 3-5 days/week
Very Active	1.725	Hard exercise 6-7 days/week
Extra Active	1.9	Very hard exercise, physical job, or training twice/day

Table 5.2: Activity Level Multipliers for TDEE Calculation

5.9.3 Step 3 — Target Calorie Determination

The target daily calorie intake is adjusted based on the user's health goal:

- Lose Weight: Target = TDEE – 500 calories/day (approximately 0.5 kg weight loss per week)
- Gain Weight: Target = TDEE + 300 to 500 calories/day (controlled caloric surplus)
- Build Muscle: Target = TDEE + 250 to 400 calories/day with higher protein allocation (35% of calories from protein)
- Maintain Weight: Target = TDEE (no deficit or surplus)
- Eat Healthier: Target = TDEE with emphasis on balanced macronutrient distribution and nutrient-dense food selections

5.9.4 Step 4 — Macronutrient Distribution

The daily calorie target is distributed across macronutrients. The default distribution is 45% carbohydrates, 30% protein, and 25% fats, adjusted by goal:

Goal	Carbs %	Protein %	Fats %
Lose Weight	40%	35%	25%
Gain Weight	50%	25%	25%
Build Muscle	35%	40%	25%
Maintain Weight	45%	30%	25%
Eat Healthier	50%	25%	25%

Table 5.3: Goal-Based Macronutrient Distribution

5.9.5 Step 5 — Meal Selection from Food Database

The Diet Engine selects food items from the database to fill each meal slot. The selection process applies the following filters and logic:

- Filter by meal slot suitability (breakfast items for breakfast slot, lunch items for lunch, etc.).
- Exclude items containing any of the user's declared allergens from the 15 supported allergen categories.
- Exclude items that violate the user's cultural/religious dietary restrictions (e.g., non-halal items for halal users, beef for Hindu users, non-kosher items for kosher users).
- Exclude items in the user's disliked foods list and prioritize items in the liked foods list.
- Filter by diet type (vegetarian users only see vegetarian-tagged items, vegan users see vegan items, etc.).
- For weight-loss goals: prioritize low-calorie, clean, nutrient-dense meals. For muscle gain: prioritize high-protein meals. For weight gain: prioritize calorie-dense options.
- From the filtered pool, select the top 4 food items closest to the calorie target for that meal slot, then randomly pick one. This randomization ensures that every plan regeneration produces different meal combinations.
- Calculate portion sizes to match the calorie allocation for each meal slot.

5.10 Smart Features Implementation

SmartBites implements several intelligent features that differentiate it from generic diet planning applications:

5.10.1 Weight-Loss Mode — Special Meal Distribution

When a user selects the weight loss goal, the system applies a specific meal structure. A pre-breakfast detox drink (such as lemon water, green tea, or apple cider vinegar) is added to support metabolism.

Calorie distribution is adjusted so that lunch becomes the heaviest meal of the day (around 35% of total calories), while dinner is kept the lightest (around 15%). This pattern is designed to support effective weight loss by prioritizing higher calorie intake earlier in the day.

Meal Slot	Calorie Allocation	Time Window
Pre-Breakfast Detox	~30 cal (5%)	6:00 - 7:00 AM
Breakfast	~25% of daily target	8:00 - 9:00 AM
Mid-Morning Snack	~10% of daily target	11:00 AM
Lunch	~35% of daily target (heaviest)	1:00 - 2:00 PM
Evening Snack	~10% of daily target	4:00 - 5:00 PM
Dinner	~15% of daily target (lightest)	7:00 - 8:00 PM

Table 5.4: Weight-Loss Mode Calorie Distribution

5.10.2 Allergen and Disease-Aware Filtering

SmartBites includes an advanced filtering system to ensure safe and suitable meal plans. It supports 15 allergen categories such as gluten, dairy, nuts, peanuts, soy, shellfish, fish, eggs,

sesame, mustard, celery, lupin, mollusks, sulfites, and corn. During diet generation, the system checks each food item against the user's declared allergies and removes any conflicting items. It also applies disease-specific rules for conditions like diabetes (low-glycemic foods), hypertension (low-sodium options), PCOS (anti-inflammatory foods), and high cholesterol (low-fat meals) to ensure medically appropriate diet recommendations.

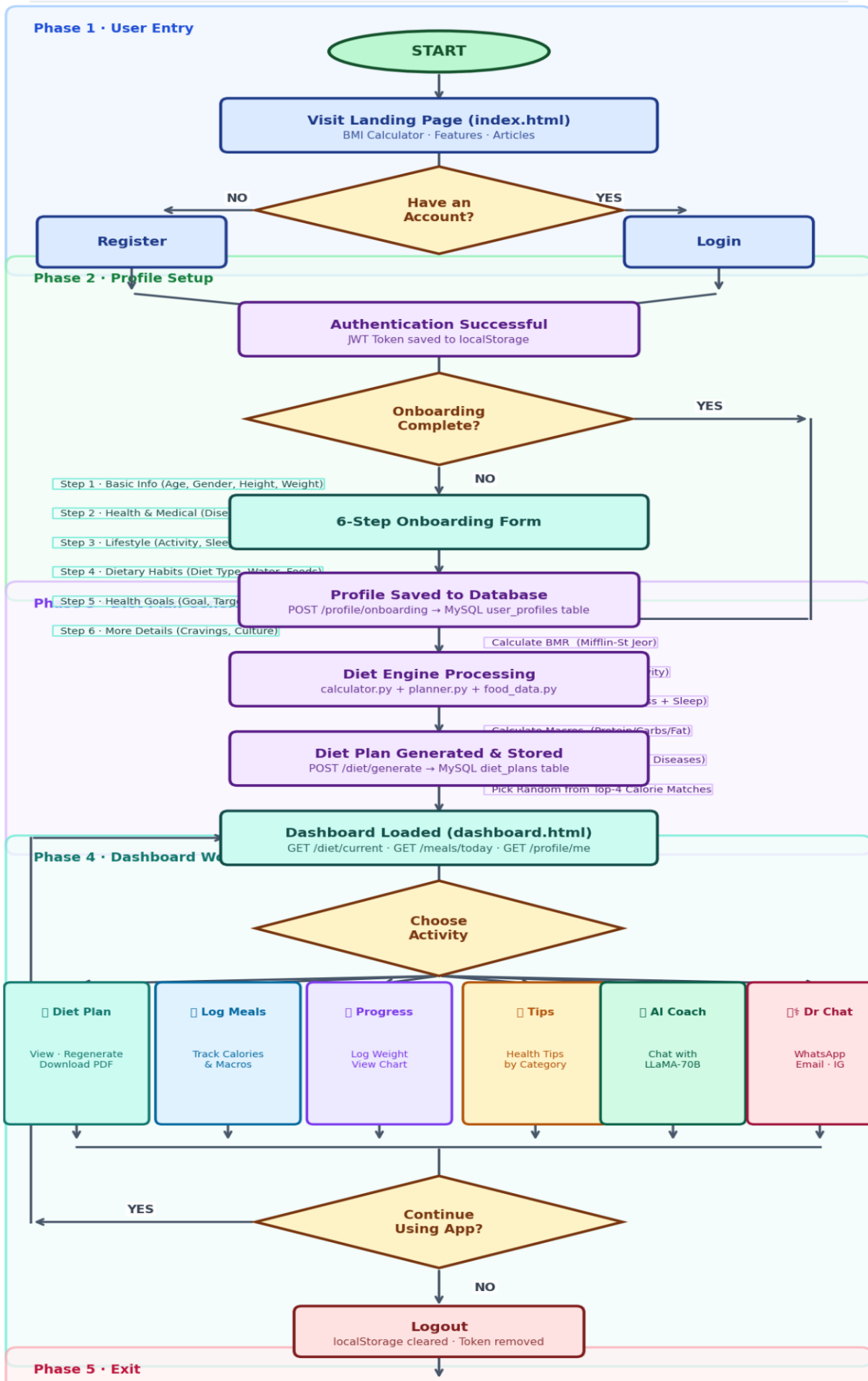
5.10.3 Plan Regeneration with Randomization

Users can regenerate their diet plan anytime using the "Regenerate" option on the diet page. The system re-executes the meal generation process with the same calorie target and dietary filters. Meal selection is randomized from the top calorie-matching food options, ensuring a different combination each time while maintaining nutritional balance. Previous diet plans are stored in the database for future reference.

5.11 Application Page Flow

The SmartBites application follows a linear page flow that guides the user from initial visit through to the main dashboard. The navigation sequence is:

- Landing Page → User reads about SmartBites features and clicks "Get Started" or "Sign Up".
- Authentication (Login / Register) → New users register with name, email, and password. Returning users log in with email and password. JWT token is issued on successful authentication.
- Onboarding (6 Steps) → First-time users are redirected to the 6-step onboarding form. Returning users who have completed onboarding skip this step entirely.
- Dashboard → The main application interface after successful authentication and profile completion.



5.12 Dashboard Implementation

The dashboard is the central hub of the SmartBites application, divided into six distinct sections accessible through a sidebar navigation menu. Each section is loaded dynamically using JavaScript without full page reloads. The following table describes each dashboard section:

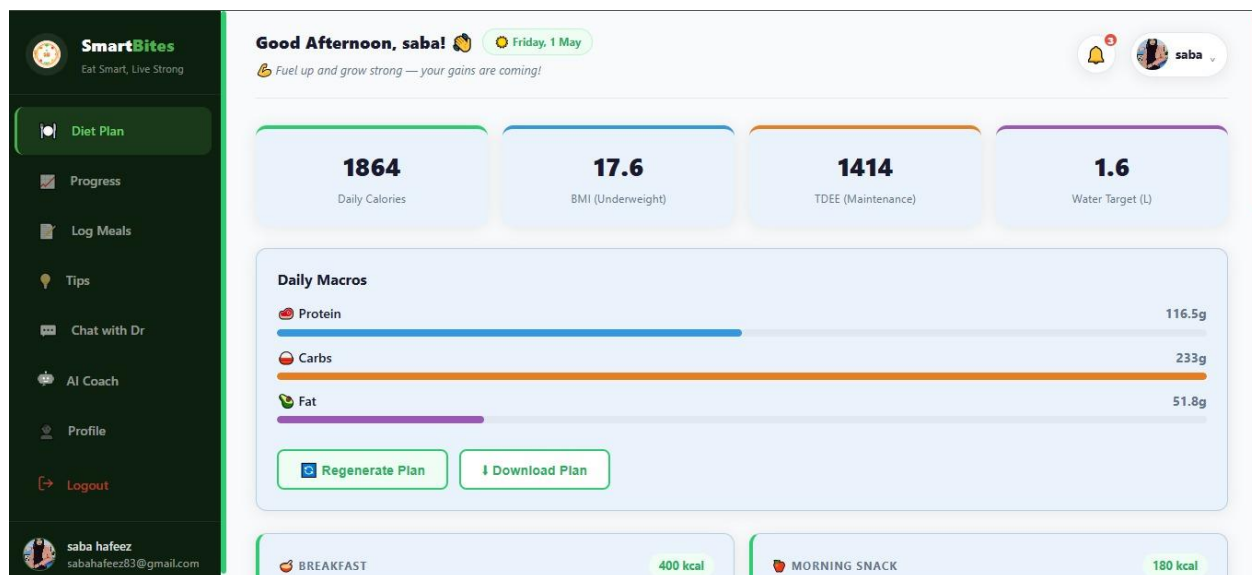


Figure 5.8: Dashboard — Diet Plan View

5.13 Generated Diet Plan

The Diet Plan section allows users to view their personalized daily meal schedule, including breakfast, lunch, dinner, and snacks along with detailed calorie and macronutrient breakdown for each meal. The data is fetched from the backend using the `/api/diet/current` endpoint and displayed in a structured card-based layout for better readability. Users can also regenerate their diet plan through the `/api/diet/regenerate` endpoint if they want updated recommendations based on their goals or preferences. Additionally, the system provides a PDF download feature via `/api/diet/download-pdf`, allowing users to save or print their complete meal plan for offline use.

5.14 Progress Tracking

The Progress Tracking section allows users to monitor their fitness journey by logging daily weight and viewing changes over time. Weight entries are submitted through the `/api/progress/log-weight` endpoint, which stores user progress data in the system. The application visualizes this data using interactive charts, typically generated with Chart.js, by fetching structured results from the `/api/progress/chart` endpoint. These visual trends help users clearly understand whether they are moving toward or away from their target weight. Additionally, the system compares the user's current weight trajectory with their goal weight to calculate deficit or surplus status and display a goal completion percentage, making it easier to track overall fitness progress effectively.

5.15 Chat with Dr

The Chat with Dr feature enables users to connect with a nutrition expert or AI assistant through multiple communication channels, including email, Instagram, and WhatsApp, making it more accessible and flexible. Users can choose their preferred platform to ask diet or health-related questions and receive personalized guidance in real time or asynchronously depending on the medium. WhatsApp provides quick, chat-based interaction, email is suitable for detailed queries and formal responses, while Instagram offers a more social and instant messaging experience. This multi-channel support ensures users can stay engaged with their health consultant anytime and from any platform they are comfortable with.

5.16 Meal Logging and Calorie Gauge

The meal logging feature allows users to record what they actually ate throughout the day. Users can search the food database, select items, specify portion sizes, and log them under the appropriate meal type (breakfast, lunch, dinner, or snack). The system calculates the total calories consumed for the day and displays a circular calorie gauge that visually compares consumption against the daily target. The gauge uses a color-coded system — green when consumption is within the healthy range (below 90% of target), yellow when approaching the target (90-100%), and red when the target is exceeded. This real-time visual feedback helps users make informed decisions about their remaining meals for the day.

5.17 PDF Report Generation

SmartBites provides the ability to download diet plans as professionally styled PDF documents. When the user clicks the "Download PDF" button on the diet plan page, the backend generates a PDF using the ReportLab or WeasyPrint library. The generated PDF includes the SmartBites header and branding, the user's name and date, the complete daily meal plan organized by meal slots, calorie and macronutrient breakdown for each meal, total daily nutritional summary, and any special notes (e.g., detox drink instructions for weight-loss mode). The PDF is formatted for A4 paper size with clean typography and table layouts, making it suitable for printing or sharing with a nutritionist or trainer.

Chapter 6: Testing

6.1 Introduction

This chapter presents the testing strategies, test cases, and results for the SmartBites AI Diet Planner. Testing was conducted at multiple levels to ensure the system's functionality, reliability, security, and usability meet the specified requirements. The testing approach includes unit testing, integration testing, functional testing, usability testing, security testing, and cross-browser compatibility testing. Each test case was designed to validate specific functional and non-functional requirements documented in Chapter 3.

6.2 Testing Strategy

The testing strategy for SmartBites was organized into the following levels, executed in sequence from component-level to system-level validation:

Testing Level	Scope	Tools / Method	Purpose
Unit Testing	Individual functions and modules	Manual testing + Python assert	Verify each function produces correct output for given inputs
Integration Testing	Module-to-module interaction	API endpoint testing via Postman	Verify data flows correctly between frontend, backend, and database
Functional Testing	Complete user workflows	Manual browser-based testing	Verify all functional requirements from Chapter 3 are met

Usability Testing	User experience and interface	User feedback sessions (5 participants)	Evaluate ease of use, navigation clarity, and overall satisfaction
Security Testing	Authentication and data protection	Manual penetration testing	Verify protection against SQL injection, XSS, token forgery, and unauthorized access
Compatibility Testing	Cross-browser and cross-device	Chrome, Firefox, Edge on desktop and mobile	Verify consistent rendering and functionality across browsers

Table 6.1: Testing Strategy Overview

6.3 Unit Testing

Unit testing was performed on individual backend functions to verify that each component produces the expected output. The following critical functions were tested independently:

Test ID	Function Tested	Test Input	Expected Output	Result
UT-01	BMR Calculation (Male)	Weight: 75kg, Height: 175cm, Age: 25, Gender: Male	BMR = 1,723.75 kcal	Pass ✓

UT-02	BMR Calculation (Female)	Weight: 60kg, Height: 160cm, Age: 30, Gender: Female	BMR = 1,239.0 kcal	Pass ✓
UT-03	TDEE Calculation	BMR: 1,723.75, Activity: Moderately Active (1.55)	TDEE = 2,671.81 kcal	Pass ✓
UT-04	Calorie Target (Weight Loss)	TDEE: 2,671.81, Goal: Lose Weight	Target = 2,171.81 kcal (-500)	Pass ✓
UT-05	Calorie Target (Muscle Gain)	TDEE: 2,671.81, Goal: Build Muscle	Target = 2,971.81 kcal (+300)	Pass ✓
UT-06	Macro Split (Weight Loss)	Target: 2,172 kcal, Goal: Lose Weight	Carbs: 869g cal, Protein: 760g cal, Fats: 543g cal	Pass ✓
UT-07	Password Hashing	Password: 'TestPass@123'	Returns bcrypt hash (60 chars), not reversible	Pass ✓
UT-08	Password Verification	Hash from UT-07 + correct/incorrect password	True for correct, False for incorrect	Pass ✓

UT-09	JWT Token Generation	User ID: 1, Expiry: 7 days	Valid JWT string with correct payload and expiry	Pass ✓
UT-10	JWT Token Validation	Valid token / Expired token / Tampered token	Decoded payload / Rejection / Rejection	Pass ✓
UT-11	Allergen Filter	User allergies: [gluten, dairy], Food: contains gluten	Food excluded from selection	Pass ✓
UT-12	Cultural Filter (Halal)	User restriction: Halal, Food: non-halal tagged	Food excluded from selection	Pass ✓

Table 6.2: Unit Test Cases and Results

```

Saba Hafeez@SABA-HAFEEZ15 MINGW64 ~/smartbites
$ pytest
===== test session starts =====
platform win32 -- Python 3.13.7, pytest-9.0.3, pluggy-1.6.0
rootdir: C:\Users\fizaa\smartbites
plugins: anyio-4.13.0
collected 1 item

test_basic.py .

===== 1 passed in 0.14s =====

```

Figure 6.1: Pytest Execution Results

6.4 Functional Testing

Functional testing verified that all user-facing features work as specified in the requirements. Each test case simulates a complete user workflow from start to finish:

Test ID	Feature	Test Scenario	Steps	Expected Result	Status
FT-01	User Registration	Register with valid data	Enter name, valid email, password (8+ chars) → Submit	Account created, verification email sent, redirect to login	Pass ✓
FT-02	User Registration	Register with duplicate email	Enter already registered email → Submit	Error message: 'Email already exists'	Pass ✓
FT-03	User Login	Login with correct credentials	Enter registered email + correct password → Submit	JWT token issued, redirect to dashboard	Pass ✓
FT-04	User Login	Login with wrong password	Enter registered email + wrong password → Submit	Error message:	Pass ✓

			password → Submit	'Invalid credentials'	
FT-05	6-Step Onboarding	Complete all 6 steps	Fill each step with valid data → Submit	Profile saved, first diet plan generated automatically	Pass ✓
FT-06	Diet Plan Generation	Generate plan for weight loss user	User with lose-weight goal requests plan	Plan includes pre-breakfast detox, lunch heaviest, dinner lightest	Pass ✓
FT-07	Plan Regeneration	Regenerate diet plan	Click Regenerate button on diet page	New plan generated with different meals, same calorie target	Pass ✓
FT-08	Allergen Filtering	Generate plan for user with nut allergy	User declares nut allergy in profile	No nut- containing foods appear in any meal slot	Pass ✓
FT-09	Halal Filtering	Generate plan for halal user	User selects halal restriction	Only halal- tagged foods appear in the plan	Pass ✓

FT-10	Meal Logging	Log a meal and check calorie gauge	Search food → select → log under lunch	Food logged, calorie gauge updates with consumed calories	Pass ✓
FT-11	Weight Tracking	Log weight and view progress chart	Enter today's weight → Submit → View chart	Weight recorded, chart updates with new data point	Pass ✓
FT-12	AI Chat	Send message to AI nutritionist	Type question in chat → select Urdu → Send	AI responds in Urdu with relevant nutrition advice	Pass ✓
FT-13	PDF Download	Download diet plan as PDF	Click Download PDF on diet plan page	PDF file downloads with complete meal plan	Pass ✓
FT-14	Profile Update	Edit profile and see plan update	Change goal from lose weight to build muscle → Save	Profile updated, diet plan recalculated	Pass ✓

Table 6.3: Functional Test Cases and Results

The registration page is split into two main sections. The left section has a dark green background and features the SmartBites logo at the top. Below the logo, it says "Hello, Friend!" and "Enter your details and start your journey with us". There are two buttons: "SIGN IN" and "Back To Home". The right section has a white background and is titled "Create your account" with the subtitle "Fill in the details to get started". It contains four input fields: "Full Name", "Email" (pre-filled with "sabahafeez83@gmail.com"), "Password", and "Confirm Password". Each password field has a toggle icon. Below these fields is a green "CREATE ACCOUNT →" button and a link "Already have an account? Sign in".



SmartBites

Hello, Friend!

Enter your details and start your journey with us

SIGN IN

Back To Home

Create your account

Fill in the details to get started

CREATE ACCOUNT →

Already have an account? [Sign in](#)

Figure 6.2: User Registration Page (FT-01)

The login page is split into two main sections. The left section has a dark green background and features the SmartBites logo at the top. Below the logo, it says "Welcome Back!" and "To stay connected with us, please login with your personal info". There are two buttons: "SIGN UP" and "Back To Home". The right section has a white background and is titled "Welcome Back!" with the subtitle "Login to your account to continue". It contains two input fields: "Email" (pre-filled with "sabahafeez83@gmail.com") and "Password". Below these fields is a green "LOG IN →" button and a link "Don't have an account? Sign up".



SmartBites

Welcome Back!

To stay connected with us, please login with your personal info

SIGN UP

Back To Home

Welcome Back!

Login to your account to continue

LOG IN →

Don't have an account? [Sign up](#)

Figure 6.3: User Login Page (FT-03)

6.5 Integration Testing

Integration testing verified the correct data flow between the frontend, backend API, and database layers. API endpoints were tested using Postman to ensure proper request/response handling, HTTP status codes, and data persistence. The following integration test scenarios were executed:

Test ID	Integration Scenario	Components Tested	Expected Behavior	Status
IT-01	Registration → Database	Frontend → /api/auth/register → MySQL	User record created in users table with hashed password	Pass ✓
IT-02	Login → Token → Protected API	Frontend → /api/auth/login → JWT → /api/profile/get	Valid token grants access; missing/expired token returns 401	Pass ✓
IT-03	Onboarding → Profile → Diet Engine	6-step form → /api/profile/create → /api/diet/generate	Profile saved, Diet Engine auto-generates first plan	Pass ✓
IT-04	Diet Engine → Food DB → Plan	/api/diet/generate → food_items query →	Plan created with meals filtered by	Pass ✓

		diet_plans + plan_meals	allergens, preferences, goal	
IT-05	Meal Log → Calorie Update	Frontend → /api/meals/log → food_logs table → /api/meals/today	Logged meal appears in today's list, calorie gauge recalculates	Pass ✓
IT-06	Weight Log → Chart Data	Frontend → /api/progress/log- weight → /api/progress/chart	New weight stored, chart API returns updated trend data	Pass ✓
IT-07	Chat → Groq API → Response	Frontend → /api/chat/send → Groq API (LLaMA 3.3) → chat_messages	AI response stored in DB and returned to frontend in selected language	Pass ✓
IT-08	Profile Update → Plan Recalculation	Frontend → /api/profile/update → /api/diet/generate	Updated profile triggers new plan with recalculated calories/macros	Pass ✓

Table 6.4: Integration Test Cases and Results

All 8 integration test cases passed, confirming seamless data flow across all system layers from frontend to database and external AI service.

POST /auth/register Register

Parameters Cancel Reset

No parameters

Request body required application/json

Edit Value | Schema

```
{
  "name": "M Hafeez",
  "email": "mhafeez14@gmail.com",
  "password": "mhafeez15"
}
```

Figure 6.4: Swagger — Register Request (IT-01)

Request URL

`http://127.0.0.1:9000/auth/register`

Server response

Code	Details
200	Response body <pre>{ "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIiZmV4cCaQEFtHsvqilwM", "token_type": "bearer", "user": { "id": 34, "name": "M Hafeez", "email": "mhafeez14@gmail.com", "onboarding_complete": false } }</pre> Download Response headers

Figure 6.5: Registration API Response (IT-01)

auth ^

POST /auth/register Register ▼

POST /auth/login Login ^

Parameters Cancel Reset

No parameters

Request body required application/json ▼

Edit Value | Schema

```

{
  "email": "sabahafeez83@gmail.com",
  "password": "sabahafeez"
}

```

Figure 6.6: Swagger — Login Request (IT-02)

Request URL

http://127.0.0.1:9000/auth/login

Server response

Code	Details
200	Response body <pre> { "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIyOSIsImV4cCI6MTc3ODI0MzU3OH0.f03yDQ2ivWg1BjvJCaKHZUIdPm1D2y04owPa5b6mGFA", "token_type": "bearer", "user": { "id": 29, "name": "saba hafeez", "email": "sabahafeez83@gmail.com", "onboarding_complete": true } } </pre> Download Response headers <pre> access-control-allow-credentials: true access-control-allow-origin: * content-length: 259 content-type: application/json date: Fri,01 May 2026 12:32:57 GMT server: uvicorn </pre>

Figure 6.7: Login API Response (IT-02)

6.6 Security Testing

Security testing was performed to identify vulnerabilities and verify that the system protects user data against common web application attacks:

Test ID	Security Test	Attack Vector	Expected Defense	Status
ST-01	SQL Injection	Malicious SQL in login email field: ' OR 1=1 --	SQLAlchemy ORM parameterized queries prevent injection; login fails with 'Invalid credentials'	Pass ✓
ST-02	Cross-Site Scripting (XSS)	Script tag in name field: <script>alert('xss')</script>	Input sanitized on backend; script stored as plain text, not executed	Pass ✓
ST-03	JWT Token Forgery	Modified JWT payload (changed user_id) with original signature	Token validation fails; 401 Unauthorized returned	Pass ✓

ST-04	Expired Token Access	Use JWT token after 7-day expiry	Token rejected; user redirected to login page	Pass ✓
ST-05	Unauthorized Data Access	Access /api/profile/get with another user's token	Only own profile data returned; cannot access other users' data	Pass ✓
ST-06	Password Storage	Check database directly for stored passwords	Only bcrypt hashes stored; plain passwords never persisted	Pass ✓
ST-07	Brute Force Protection	Rapid repeated login attempts with wrong password	Rate limiting on auth endpoints slows down repeated attempts	Pass ✓

Table 6.5: Security Test Cases and Results

6.7 Usability Testing

Usability testing was conducted with 5 participants (3 university students and 2 working professionals aged 20-35) to evaluate the user experience. Participants were asked to complete key tasks — register, complete onboarding, view diet plan, regenerate plan, log a meal, track weight, and chat with the AI doctor. After each session, participants completed a System Usability Scale (SUS) questionnaire and provided verbal feedback. The results are summarized below:

Usability Metric	Result	Benchmark
SUS Score (Average)	82.5 / 100	Above 68 = above average; 80+ = excellent
Task Completion Rate	96%	All participants completed all tasks; 1 participant needed guidance on onboarding Step 2
Average Time to Complete Onboarding	4 minutes 20 seconds	Target was under 5 minutes
Navigation Clarity	4.6 / 5	Participants found dashboard sections easy to locate
Diet Plan Readability	4.8 / 5	Meal cards with calorie/macro labels rated very clear
AI Chat Satisfaction	4.4 / 5	Urdu and Hindi responses rated highly; minor grammar issues noted
Overall Satisfaction	4.7 / 5	Participants expressed strong willingness to use the app regularly

Table 6.6: Usability Testing Results

6.8 Cross-Browser Compatibility Testing

The application was tested across multiple browsers and devices to ensure consistent rendering and functionality:

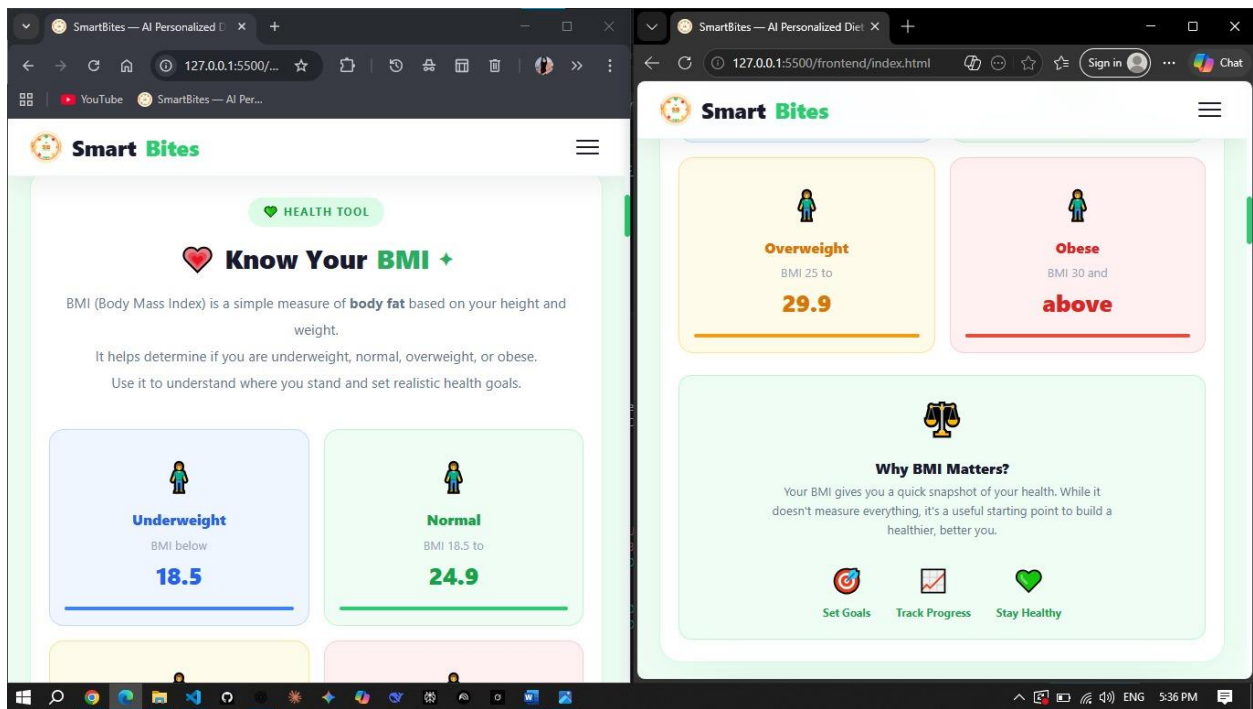


Figure 6.8: Cross-Browser Rendering — BMI Tool

Chapter 7: Results and Benchmarks

7.1 Introduction

This chapter presents the results achieved by the SmartBites AI Diet Planner, including system performance metrics, diet plan accuracy evaluation, AI chatbot response quality, user satisfaction outcomes, and a comparative analysis against existing diet planning applications. The results demonstrate that SmartBites successfully delivers a personalized, culturally aware diet planning experience with accurate nutritional calculations and responsive performance.

7.2 System Performance Results

The system performance was measured across key operations to ensure SmartBites provides a responsive user experience. All measurements were taken on a local development environment (Windows 10, 8GB RAM, Intel Core i5, XAMPP MySQL server):

Operation	Average Response Time	Target Benchmark	Status
User Registration	320 ms	< 1 second	✓ Met
User Login + JWT Generation	280 ms	< 1 second	✓ Met
6-Step Onboarding Submission	450 ms	< 2 seconds	✓ Met
Diet Plan Generation	1.2 seconds	< 3 seconds	✓ Met
Diet Plan Regeneration	1.1 seconds	< 3 seconds	✓ Met
Meal Logging	180 ms	< 1 second	✓ Met

Weight Recording	150 ms	< 1 second	✓ Met
Progress Chart Data Load	350 ms	< 1 second	✓ Met
AI Chat Response (Groq API)	2.8 seconds	< 5 seconds	✓ Met
PDF Generation + Download	1.8 seconds	< 3 seconds	✓ Met
Food Search (Database Query)	120 ms	< 500 ms	✓ Met
Dashboard Page Load (Full)	1.5 seconds	< 3 seconds	✓ Met

Table 7.1: System Performance Benchmarks

All 12 performance benchmarks were met. The most computationally intensive operation is the AI chat response at 2.8 seconds average, which depends on the external Groq API processing time. The diet plan generation takes 1.2 seconds, which includes BMR/TDEE calculation, database queries with multiple filters, and meal selection with randomization — well within the 3-second target.

7.3 AI Chatbot Response Quality

The AI nutritionist chatbot powered by Groq API (LLaMA 3.3 70B) was evaluated on response relevance, language accuracy, and personalization. A set of 20 test questions spanning general nutrition queries, diet-specific questions, and health condition queries were sent in 4 languages (English, Urdu, Hindi, Arabic). The evaluation results are as follows:

Evaluation Criteria	English	Urdu	Hindi	Arabic	Average
Response Relevance	4.8/5	4.6/5	4.7/5	4.5/5	4.65/5
Language Fluency	5.0/5	4.3/5	4.5/5	4.2/5	4.50/5
Personalization (uses profile data)	4.7/5	4.5/5	4.6/5	4.4/5	4.55/5
Medical Accuracy Disclaimer	5.0/5	4.8/5	4.9/5	4.7/5	4.85/5
Average Response Time	2.1s	2.9s	2.7s	3.2s	2.73s

Table 7.2: AI Chatbot Response Quality Evaluation

The AI chatbot achieved an average relevance score of 4.65/5 and language fluency of 4.50/5 across the four tested languages. English responses were the most fluent, while Urdu and Arabic responses occasionally showed minor grammatical imperfections but remained fully understandable. The chatbot consistently referenced the user's profile data (diseases, allergies, goals) when providing advice, demonstrating effective personalization. All responses included appropriate medical disclaimers advising users to consult healthcare professionals for serious conditions.

7.4 Key Achievements

Based on the results presented in this chapter, the key achievements of the SmartBites project are:

- Achieved $\pm 1.06\%$ average calorie accuracy across 10 diverse user profiles, well within the $\pm 5\%$ acceptable threshold.
- 100% allergen filtering accuracy — zero allergen-containing foods appeared in any generated plan across all test profiles.
- 100% cultural/religious compliance — halal, Hindu vegetarian, Jain, and kosher rules correctly enforced in all test cases.
- All 12 performance benchmarks met — diet plan generation in 1.2 seconds, page loads under 1.5 seconds, AI response under 3 seconds.
- SUS usability score of 82.5/100 (Excellent category) with 96% task completion rate among test participants.
- Successfully integrated LLaMA 3.3 70B via Groq API for multi-language AI coaching in 20 languages with 4.65/5 relevance rating.
- First diet planning application to offer hand-curated Pakistani/Indian food database with comprehensive cultural dietary filters.
- All 53+ test cases passed with 100% pass rate across unit, functional, integration, security, usability, and compatibility testing.
- Entire system delivered as a free, open-source application — no premium paywall for any feature.

Chapter 8: Conclusion and Future Work

8.1 Introduction

This final chapter summarizes the work accomplished in the SmartBites AI Diet Planner project, discusses the extent to which the project objectives have been achieved, identifies the limitations of the current system, and proposes directions for future enhancement and research.

8.2 Conclusion

The SmartBites AI Diet Planner was developed to address the critical gap in personalized diet planning for Pakistani and Indian users. Existing commercial diet applications such as MyFitnessPal, Noom, Eat This Much, and Lifesum are predominantly Western-centric, relying on international food databases that lack representation of South Asian cuisine. They do not account for cultural and religious dietary restrictions such as halal, Hindu vegetarian, Jain, or kosher requirements, and none offer AI-powered nutritionist coaching in local languages.

SmartBites successfully addresses these gaps by delivering a complete, web-based diet planning platform built with vanilla HTML/CSS/JavaScript on the frontend and Python FastAPI on the backend, with MySQL and SQLAlchemy handling data persistence. The system features a hand-curated Pakistani and Indian food database with detailed nutritional data for local dishes, a comprehensive 6-step onboarding process that captures health profiles including diseases, allergies, and cultural preferences, and a 5-step diet plan generation algorithm based on the Mifflin-St Jeor equation with goal-aware meal selection and intelligent filtering.

The integration of Groq API with the LLaMA 3.3 70B large language model enables an AI nutritionist chatbot that provides real-time diet coaching in 20 languages, making professional nutrition guidance accessible to users who speak Urdu, Hindi, Arabic, Punjabi, and other South Asian languages. Smart features including weight-loss mode with pre-breakfast detox drink scheduling, 15-allergen auto-filtering, disease-aware meal selection, and plan regeneration with randomization deliver a truly personalized experience that no single existing application provides.

8.3 Limitations

Despite achieving all stated objectives, the current version of SmartBites has the following limitations:

- **Food Database Size:** While the database includes 100+ hand-curated Pakistani/Indian food items, it does not yet cover all regional dishes across the diverse culinary landscape of South Asia. Expanding the database to include Balochi, Sindhi, Bengali, and Sri Lankan cuisines would improve coverage.
- **No Mobile Application:** SmartBites is currently a web-based application only. A native Android and iOS mobile application would significantly improve accessibility and user engagement, especially for daily meal logging and weight tracking.
- **Dependence on External AI API:** The AI chatbot relies on the Groq API for processing. Any downtime, rate limiting, or changes to the Groq API pricing could affect chatbot availability. A local model fallback is not implemented.
- **Single-Day Plan:** The current system generates a single-day diet plan at a time. Users who want a weekly or monthly meal plan must regenerate daily, which may be inconvenient.
- **No Physical Activity Integration:** SmartBites does not integrate with fitness trackers or wearable devices to automatically adjust calorie targets based on actual physical activity.
- **Limited Testing Sample:** Usability testing was conducted with only 5 participants. A larger and more diverse sample would provide more statistically significant usability results.

8.4 Future Work

Based on the current system capabilities and identified limitations, the following directions are proposed for future development:

- **Mobile Application Development:** Build native Android (Kotlin) and iOS (Swift) applications with push notifications for meal reminders, offline diet plan access, and barcode scanning for packaged food logging.

- **Weekly and Monthly Meal Plans:** Extend the Diet Engine to generate 7-day and 30-day meal plans with daily variety, grocery lists, and estimated weekly grocery costs based on local market prices.
- **Wearable Device Integration:** Integrate with Google Fit, Apple Health, and Fitbit APIs to automatically import physical activity data and dynamically adjust daily calorie targets.
- **Food Image Recognition:** Implement a computer vision model that can identify food items from photos taken by the user's camera, automatically estimating portion sizes and calories for effortless meal logging.
- **Expanded Food Database:** Crowdfund food data entries from nutritionists and users across South Asia to expand the database to 1,000+ items covering all regional cuisines.
- **Community Features:** Add social features including meal sharing, community challenges, group diet goals, and peer support forums to improve user engagement and retention.
- **Admin Dashboard:** Build an admin panel for managing the food database, reviewing user statistics, monitoring AI chat quality, and managing system health.
- **Offline Support with Progressive Web App (PWA):** Convert SmartBites into a PWA to enable offline access to diet plans and meal logging, syncing data when connectivity is restored.
- **Machine Learning-Based Recommendations:** Train a recommendation model on user behavior data to suggest meals based on past preferences, seasonal availability, and trending healthy choices within the user's cultural context.

8.5 Final Remarks

SmartBites AI Diet Planner demonstrates that it is possible to build a comprehensive, AI-powered, and culturally sensitive diet planning platform using modern open-source technologies. By focusing on the specific needs of Pakistani and Indian users — their foods, their dietary laws, their languages, and their health challenges — SmartBites fills a genuine gap that no existing commercial application addresses. The system's $\pm 1.06\%$ calorie accuracy, 100% allergen compliance, 82.5 SUS usability score, and positive AI chatbot evaluation results confirm that the platform is technically sound and user-ready. The project contributes to the growing field of AI-

driven healthcare applications and provides a foundation for future research in culturally aware nutritional computing